

Chapter 5. MSA Concepts and Design Principles



이영곤

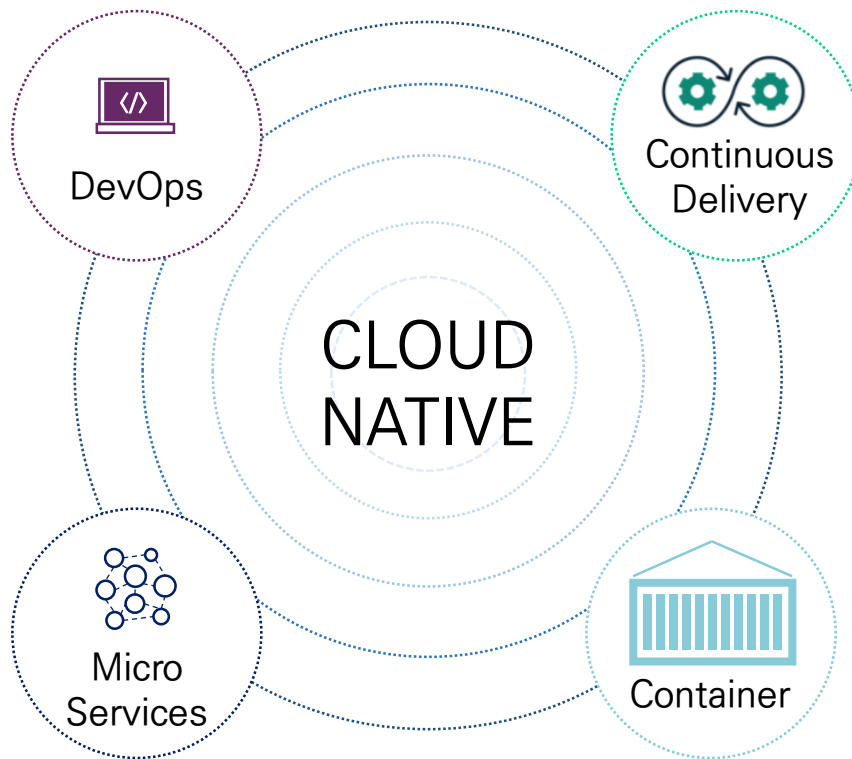


A top-down view of a wooden desk with a laptop, a cup of coffee, a pair of glasses, and a small succulent plant. The text "01. Cloud Native Application Overview" is overlaid in the center.

01. Cloud Native Application Overview

What is Cloud Native Application?

» Cloud Native



CLOUD NATIVE

An approach to developing and running applications to take advantage of the cloud computing model

Works efficiently in a cloud environment and leverages features of the cloud such as resilience, usage-based billing, disability awareness, etc

03 Micro Service Architecture

» Understanding MSA



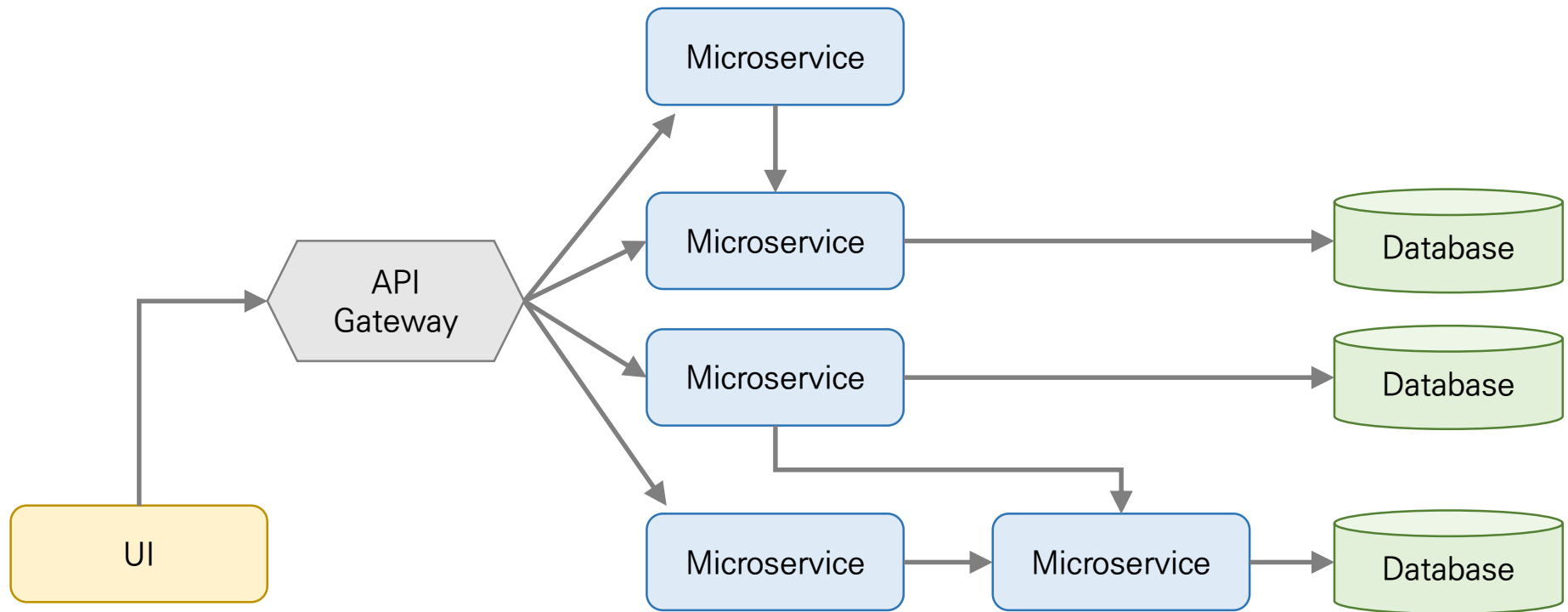
Micro Service Architecture

An architecture that decomposes and services functions into small, independent units (modules) that can be run alone and deployed independently

To apply to large-scale software development

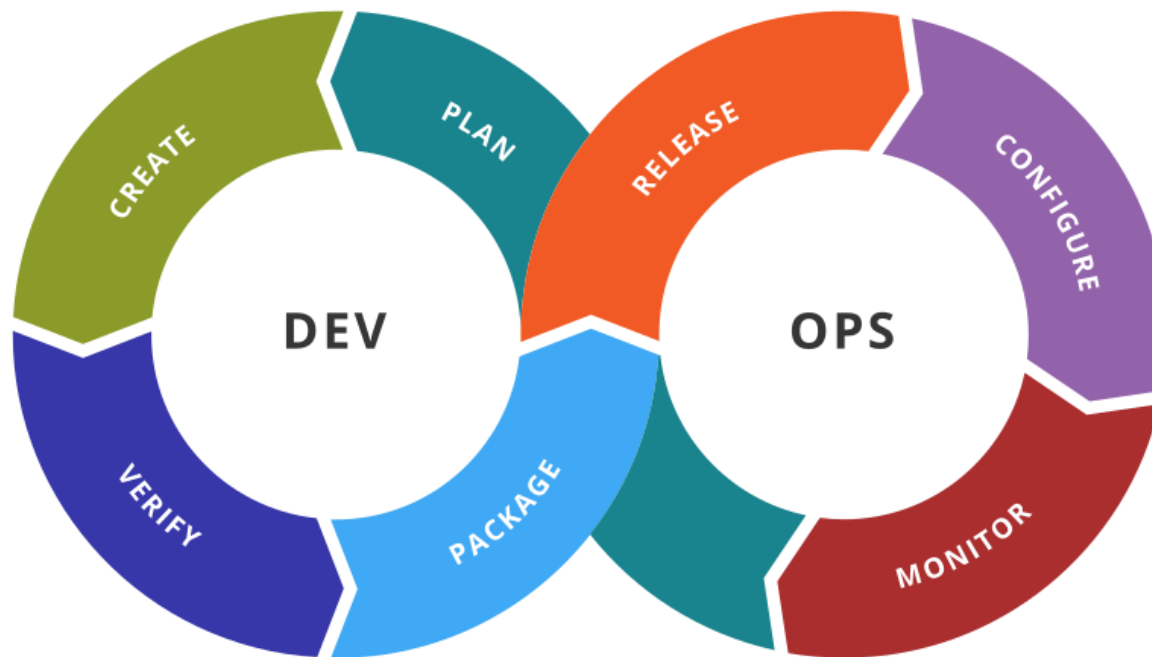
» Microservice

- ✓ An architectural approach to developing an application as a small set of services that communicate through APIs



» DevOps

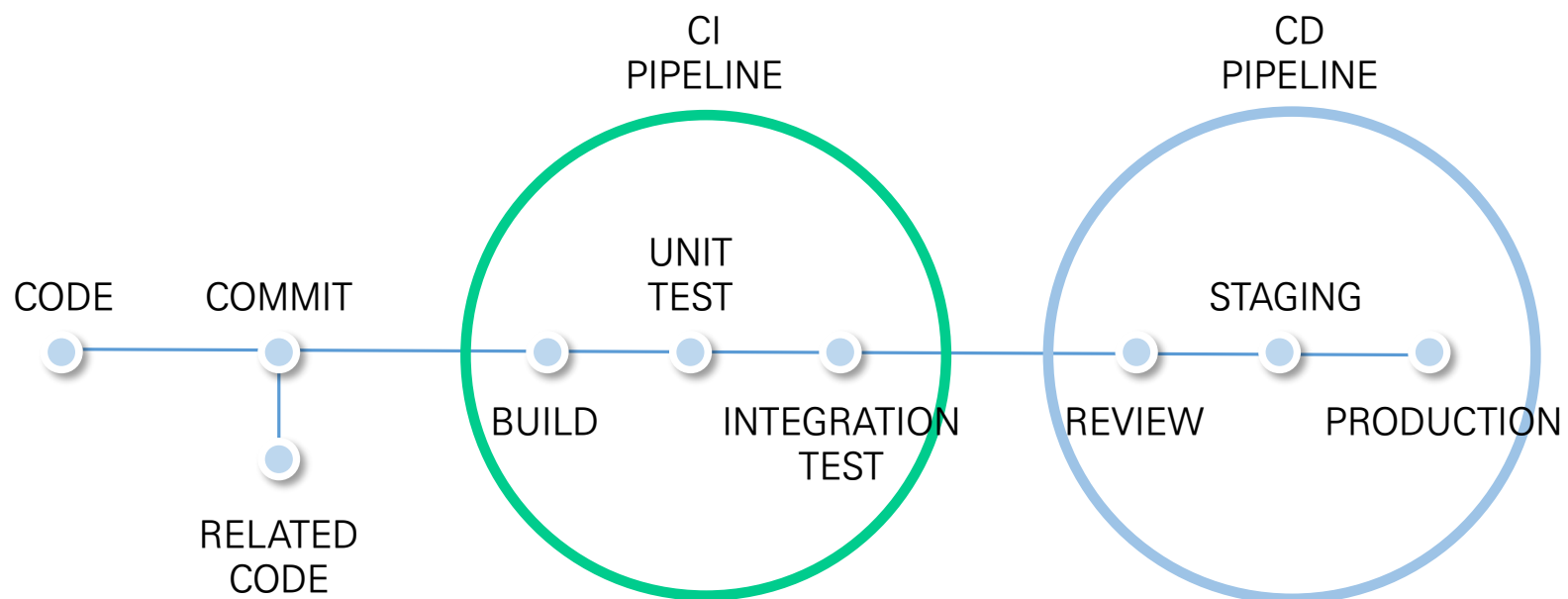
- ✓ Culture and environment that enables developers and operators to collaborate, create, test, and release reliable software in a short period of time with the goal of process automation



What is Cloud Native Application?

» Continuous Delivery

- ✓ Build and test regularly for application code changes, provide continuous integration and service to ensure reliability for frequent build/deployment



» Container

- ✓ Enables operating system-level virtualization, allocates unique processes and resources to isolated container spaces for deployment of microservices

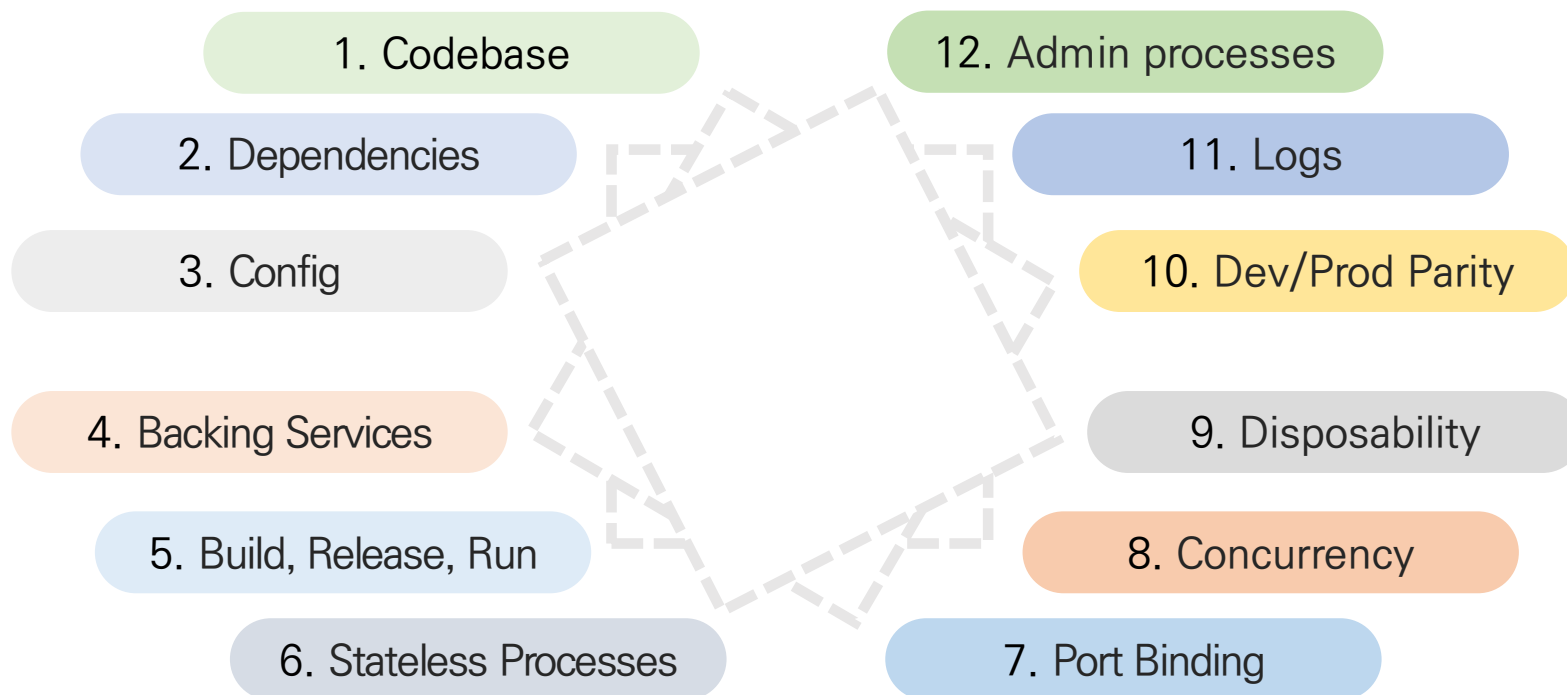


What are cloud native applications?

» Cloud Native Application Development Principles

Container Unit	a loose bond	independent/autonomous	CI/CD
Stateless	Http, Rest	Bind/Distributed DB Orientation	DevOps, automation

✓ 12 Factors

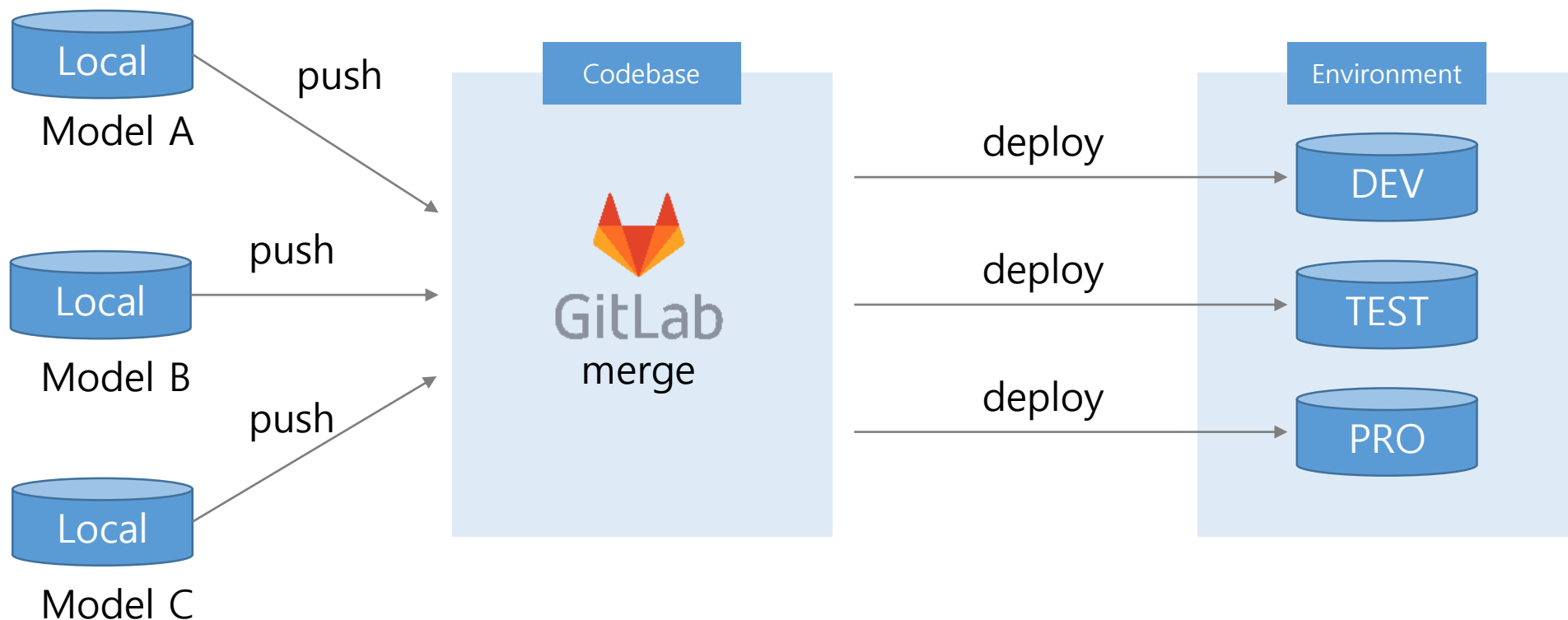


What are cloud native applications?

» Cloud Native Application Development Principles

1. Code Base

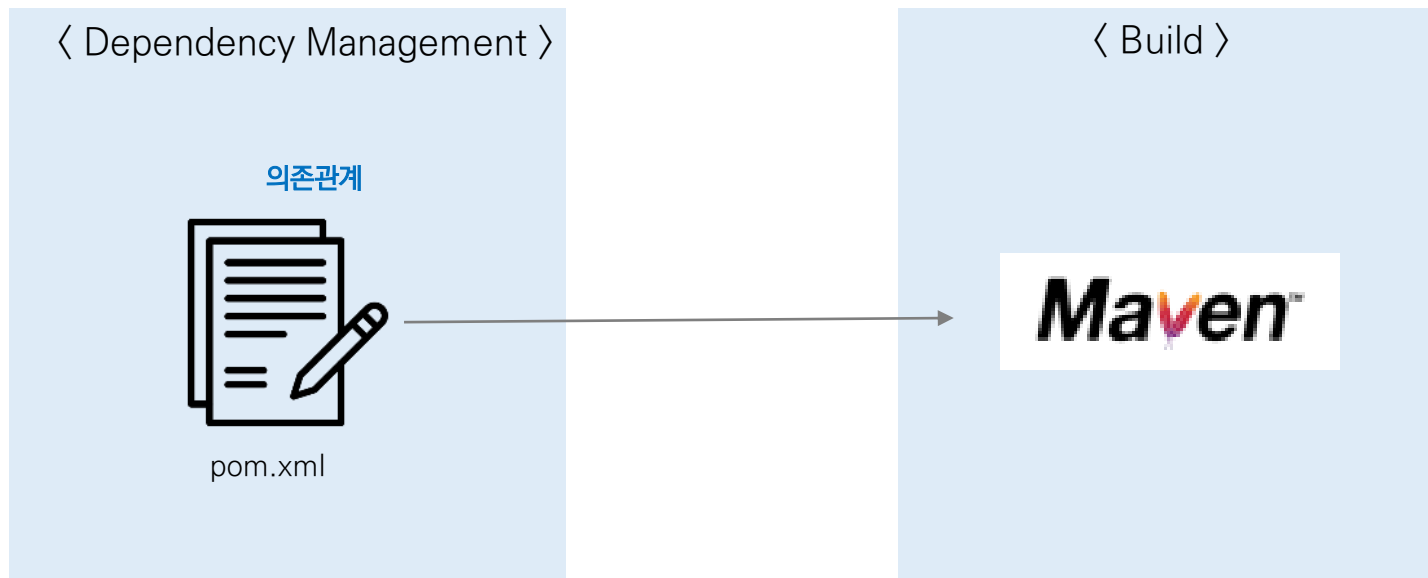
Applications must be managed through a single code base, deployed in a development and operational environment with the same code, and there is only one code base for each application (applications can be traced to changes in source code and multiple deployments to multiple environments).



» Cloud Native Application Development Principles

2. Dependency

All applications in a dependency relationship explicitly declare dependency and use the library that the application needs in the dependency file

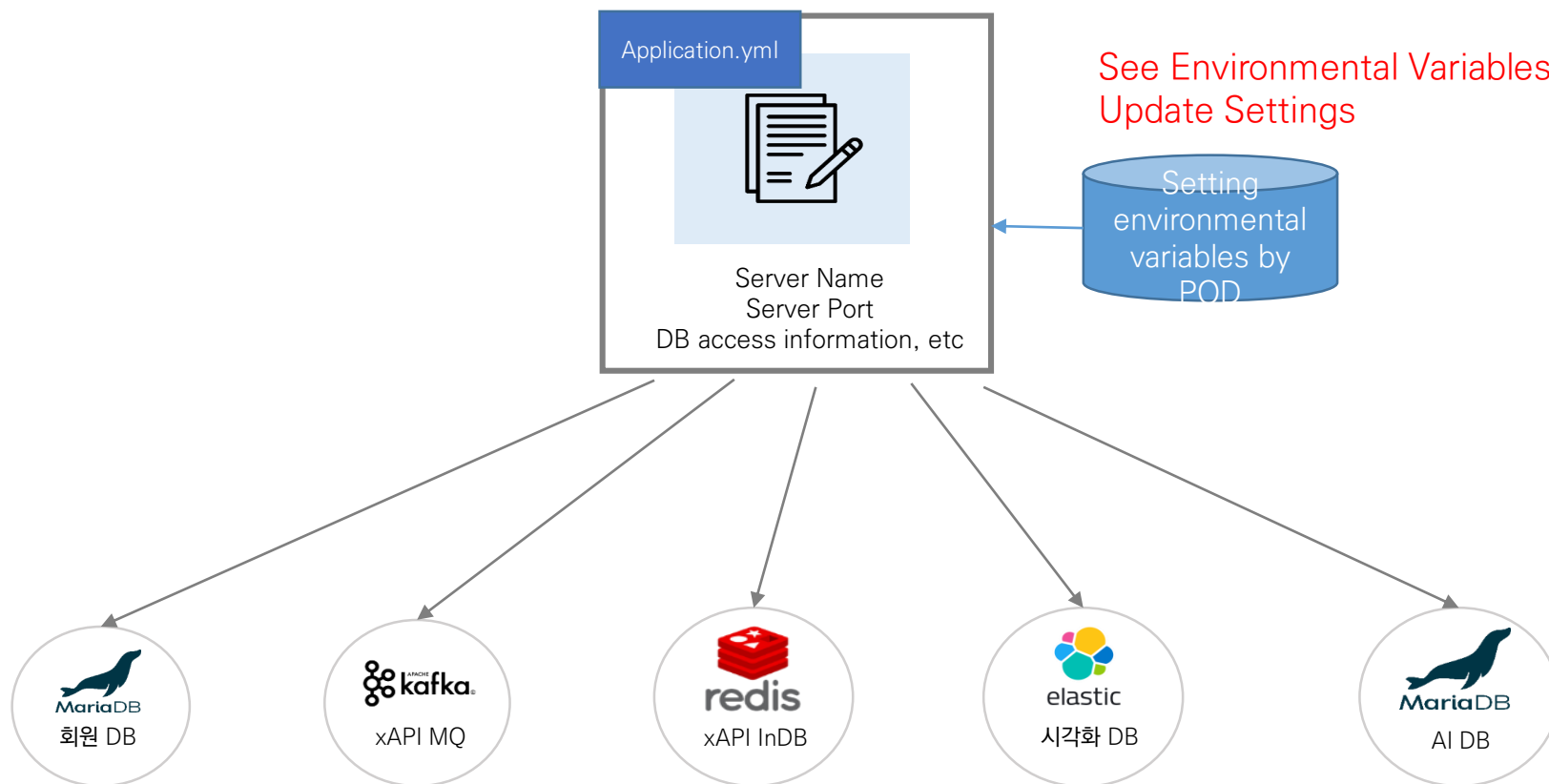


What are cloud native applications?

» Cloud Native Application Development Principles

3. Setting up

Separate configuration information and code required to run an application
(Configuration information that depends on internal and external resources, deployment stage, OS, etc. that are not related to business processing logic)

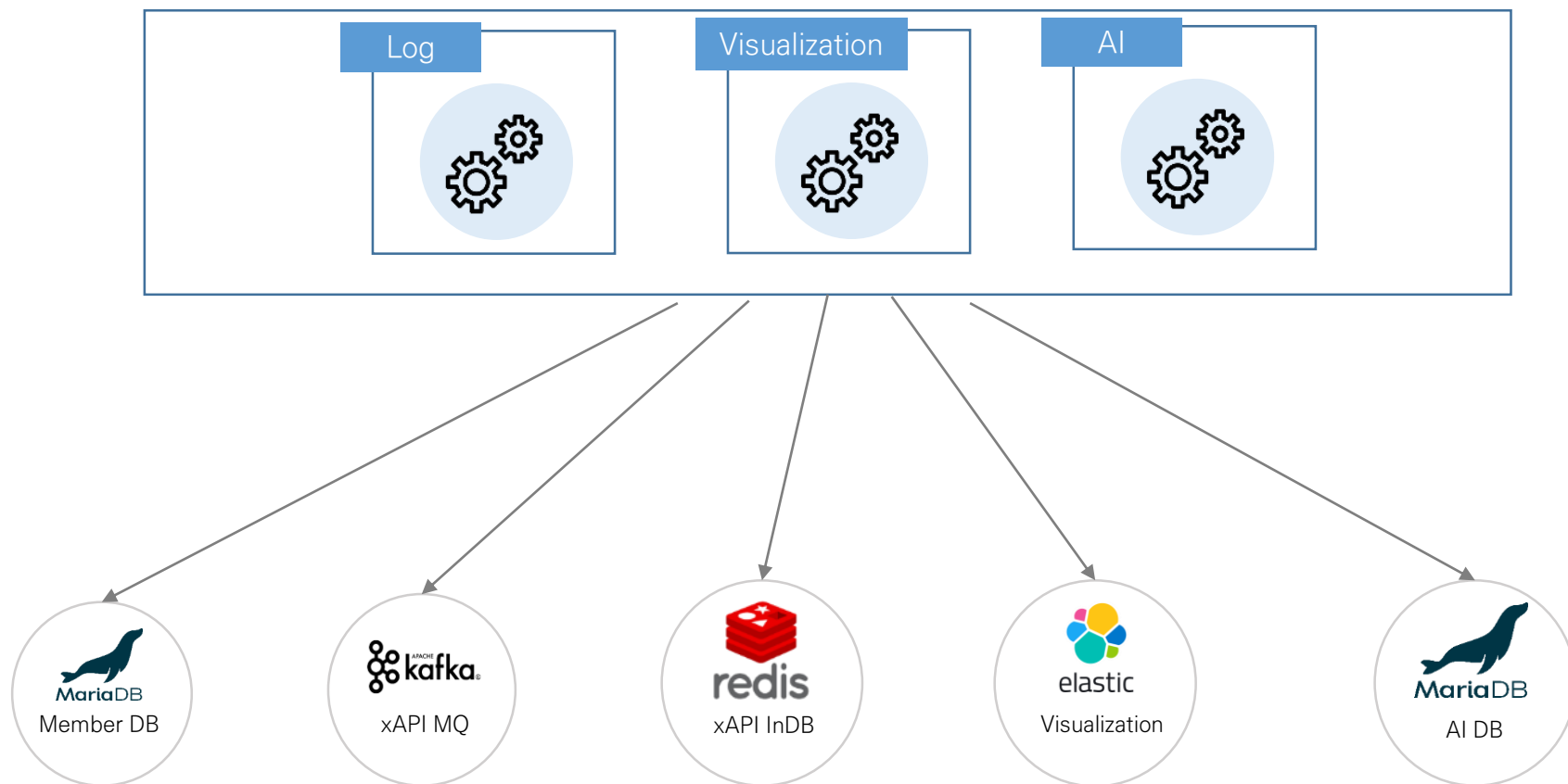


What are cloud native applications?

» Cloud Native Application Development Principles

4. back-end service

Backend services (DB, cache, message/queue, email services, SNS, cloud services, etc.) are treated as resources tied to applications

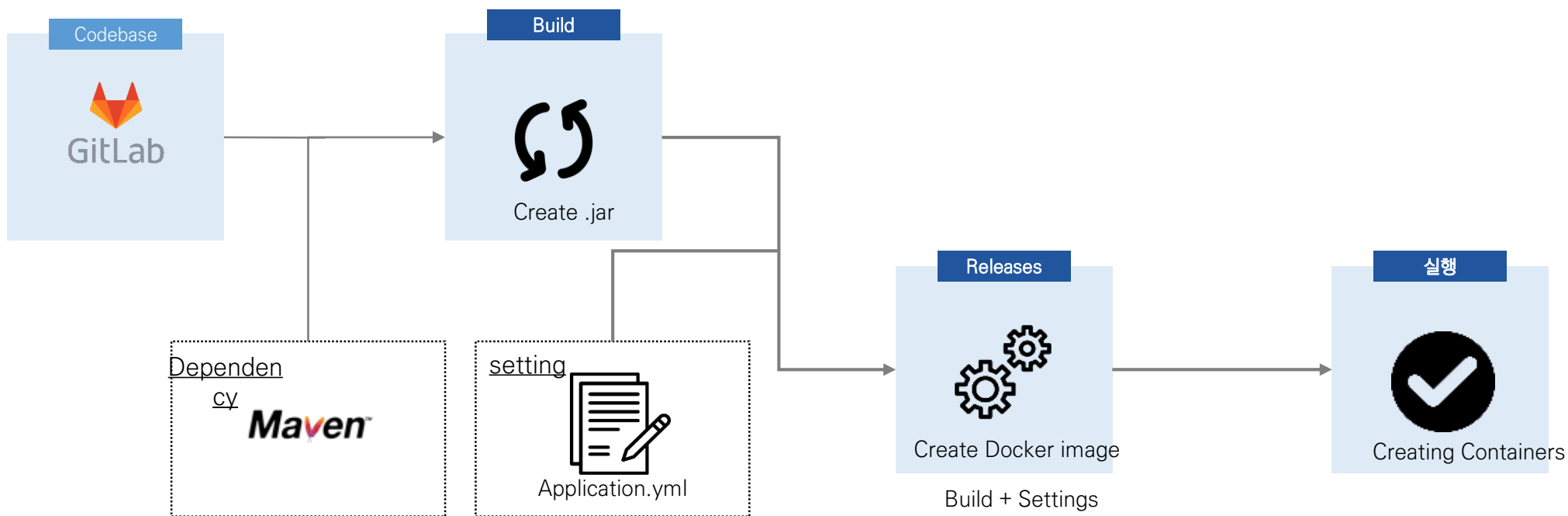


What are cloud native applications?

» Cloud Native Application Development Principles

5. build, release, run

Codebase is deployed through three stages of strictly separate build, release, and execution, each of which must be strictly separated

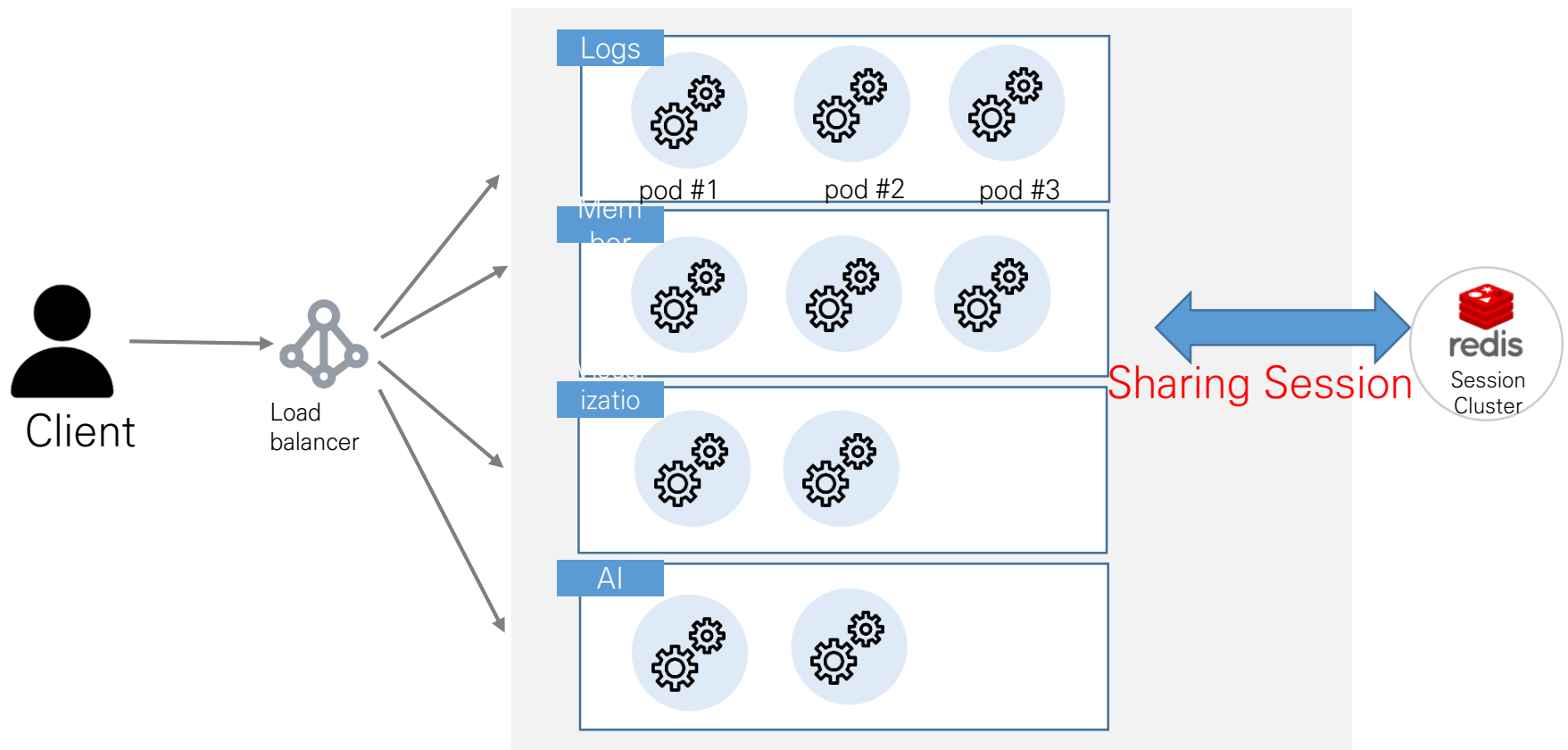


What are cloud native applications?

» Cloud Native Application Development Principles

6. Process (Statusless Service)

Store externally rather than on a server for easy scale-out structure

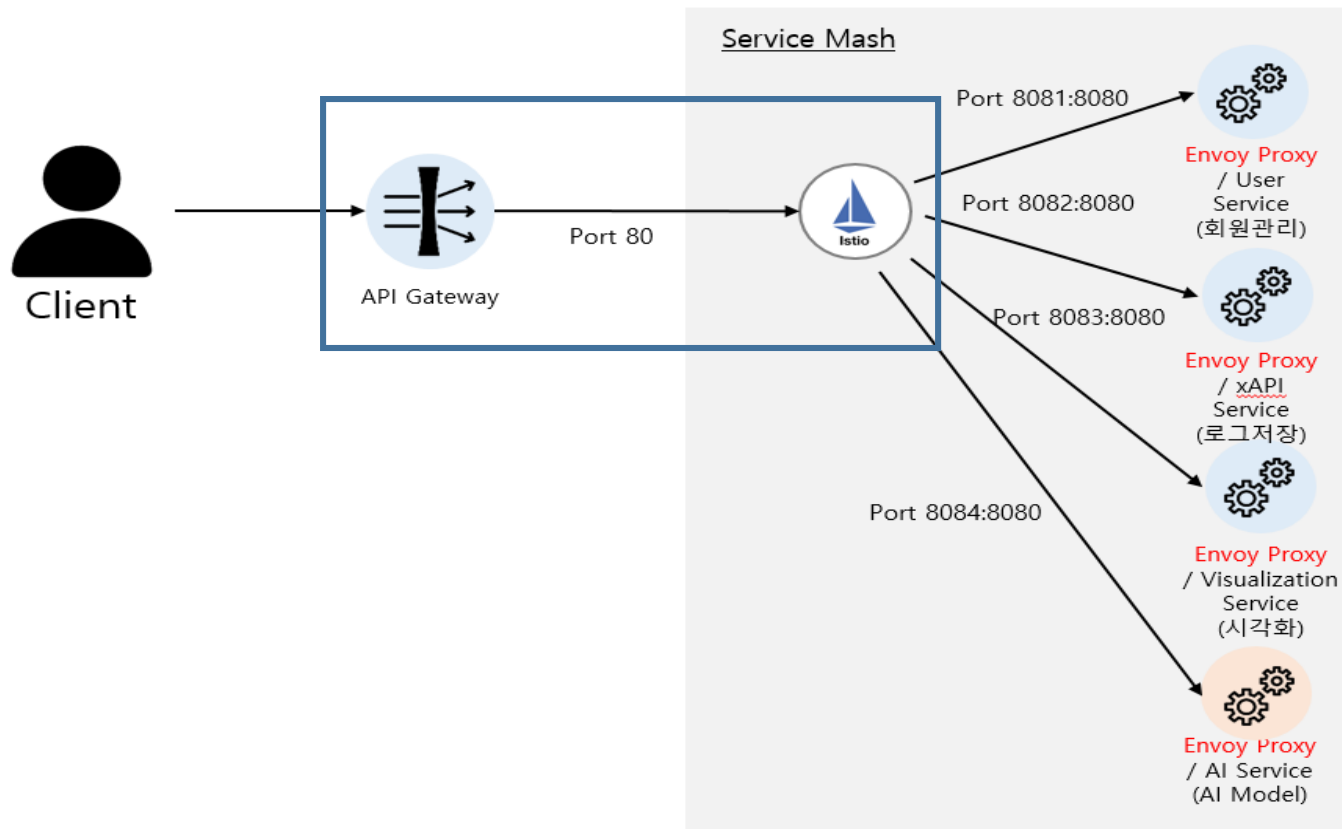


What are cloud native applications?

» Cloud Native Application Development Principles

7. Port Binding

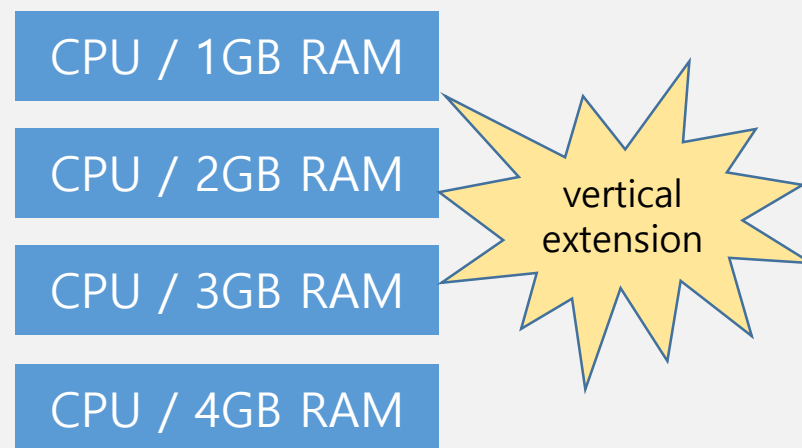
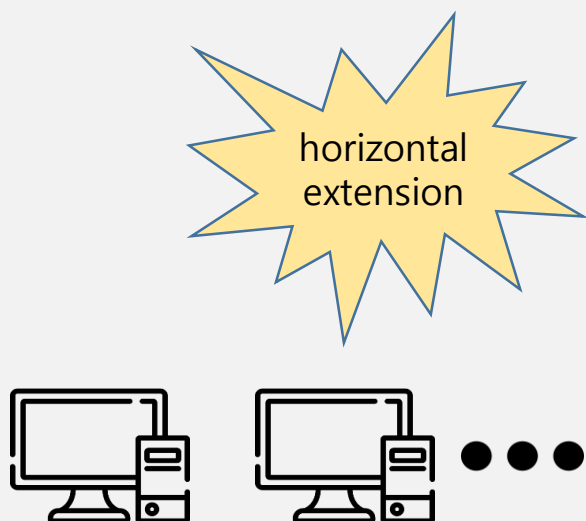
Configuration information that determines where and how messages are sent and received and services are released through port binding so that distributed applications can be accessed by other applications



» Cloud Native Application Development Principles

8. synchronicity

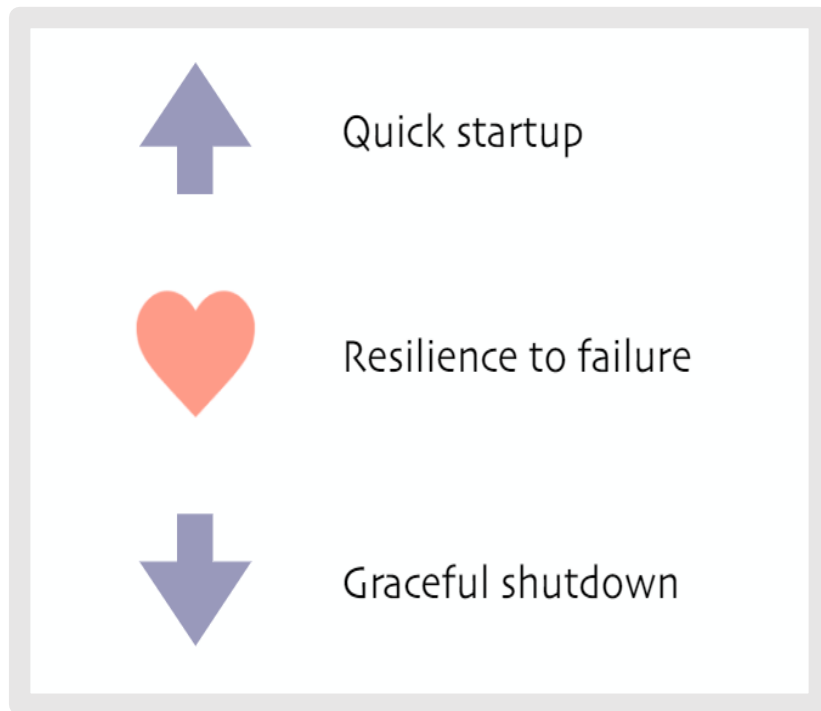
Instead of a single, all-consuming process, we run separate micro-processes by function, providing scale-up and scale-out scalability based on the process model



What are cloud native applications?

» Cloud Native Application Development Principles

9. Can be discarded



With a quick start and a graceful shutdown
Maximize stability

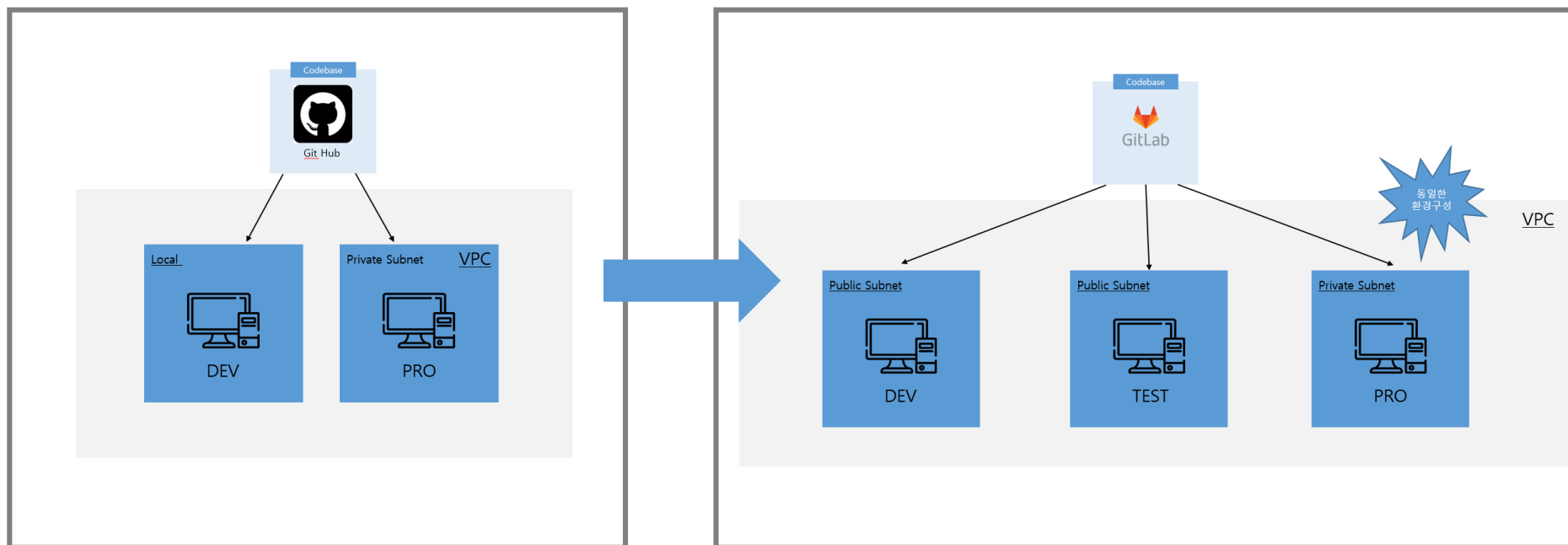
- ✓ Processes are designed to start or end immediately
- ✓ Minimize startup time, enabling agile release operations and scaling
- ✓ Graceful shutdown enables processing operations to return to queue
- ✓ Proceed to return work to the queue despite sudden shutdowns

What are cloud native applications?

» Cloud Native Application Development Principles

10. DEV / PROD Matching

Maintain the same development/verification/operational environment as possible to minimize failures and ensure continuous deployment

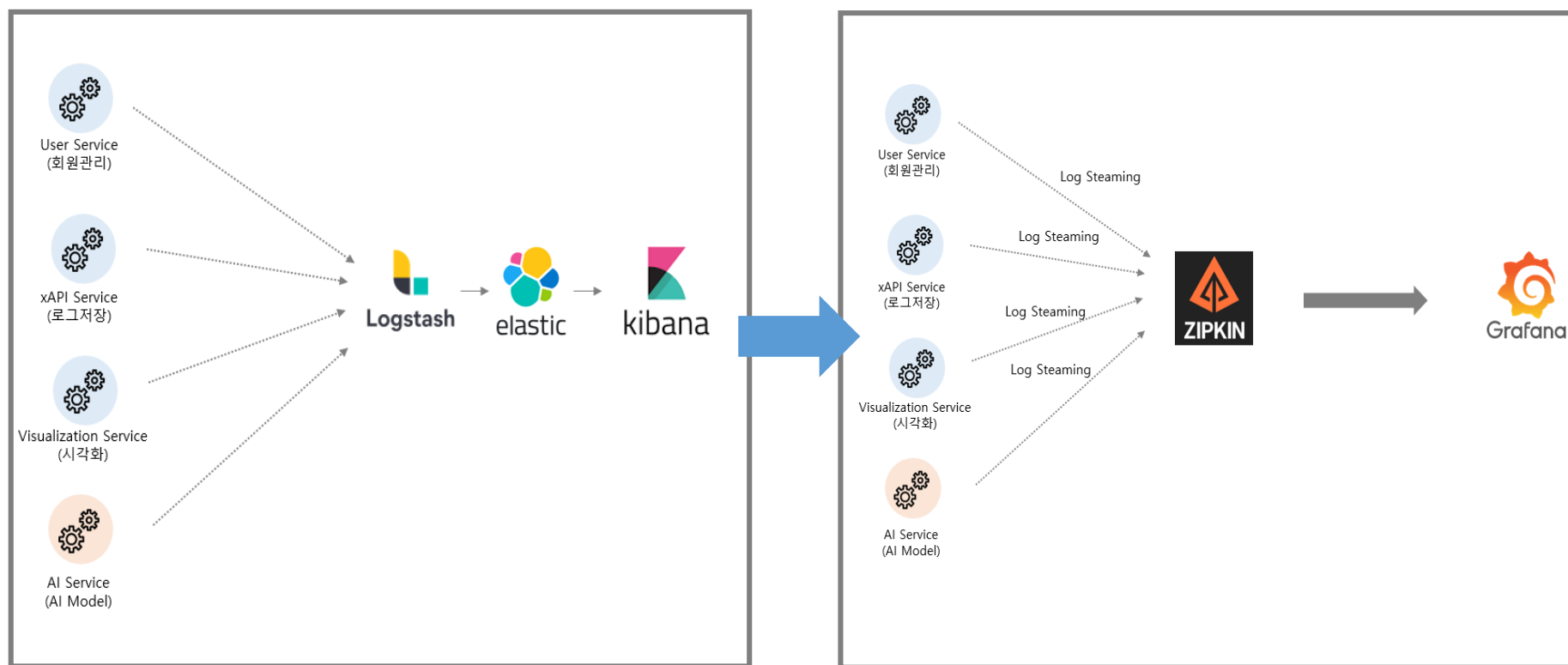


What are cloud native applications?

» Cloud Native Application Development Principles

11. Log

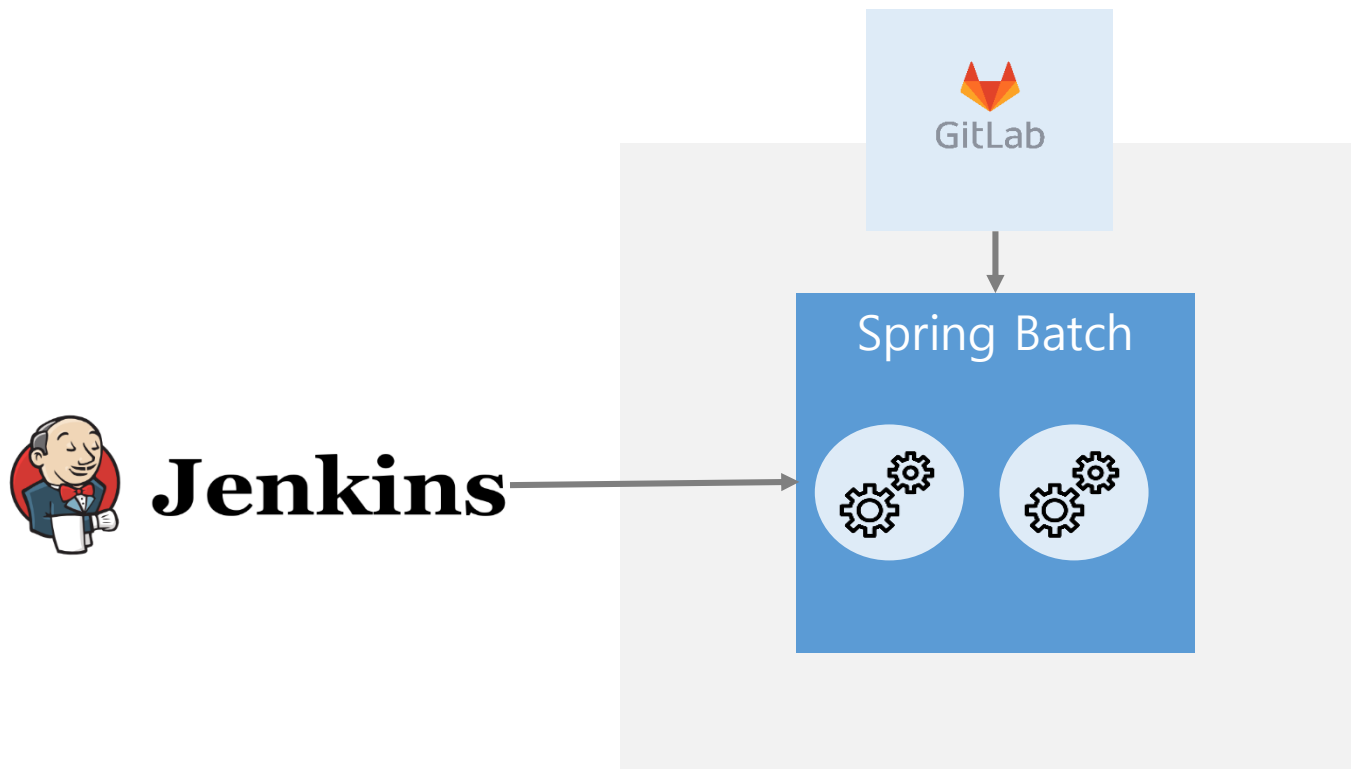
Application logs are standard output in the form of log streams rather than files, and log streams are integrated and managed by the platform



» Cloud Native Application Development Principles

12. Admin Process

Run the management/maintenance process as a one-time process, which runs separately from the application process, but in the same build/release/deployment cycle as the application



Why use Cloud Native Application?

» Why Use Cloud Native Application

Speed of Innovation

- ✓ Apply a variety of new technologies to deliver applications or deploy new versions of software
- ✓ Improper deployment requires immediate recovery

Always-Available Services(Safety)

- ✓ Limit the scope of failure-affected service configurations and minimize dependency to avoid spreading
- ✓ The system automatically recovers standardized types of failures

Web Scale

- ✓ Vertical scaling with scale-up
Horizontal scaling with low cost and fast scale-out
- ✓ Workload orchestration with virtualization of small, uniform servers

Mobile-Centric User Experiences

- ✓ Extreme expansion of diversity in IT environments and business services
- ✓ The need for mobile-oriented development is emerging due to the expansion of the base of smartphones

Why use Cloud Native Application?

» Emerging problems with existing applications

- ✓ Disadvantages for larger applications
 - Long time to build, deploy, and boot servers
 - One or two people's mistakes can lead to a build failure of the entire system, making it harder to collaborate as the project grows
 - Components are tightly coupled to each other on a local call-by-reference basis, affecting performance problems or failures in a particular component or module to other components
 - Code gets too big to maintain
 - Difficult to identify the impact of changes to existing logic, data, interfaces, etc
- ✓ Disadvantages for frequently deployed systems
 - Minor component modifications must be re-compiled and distributed throughout the application and subjected to QA

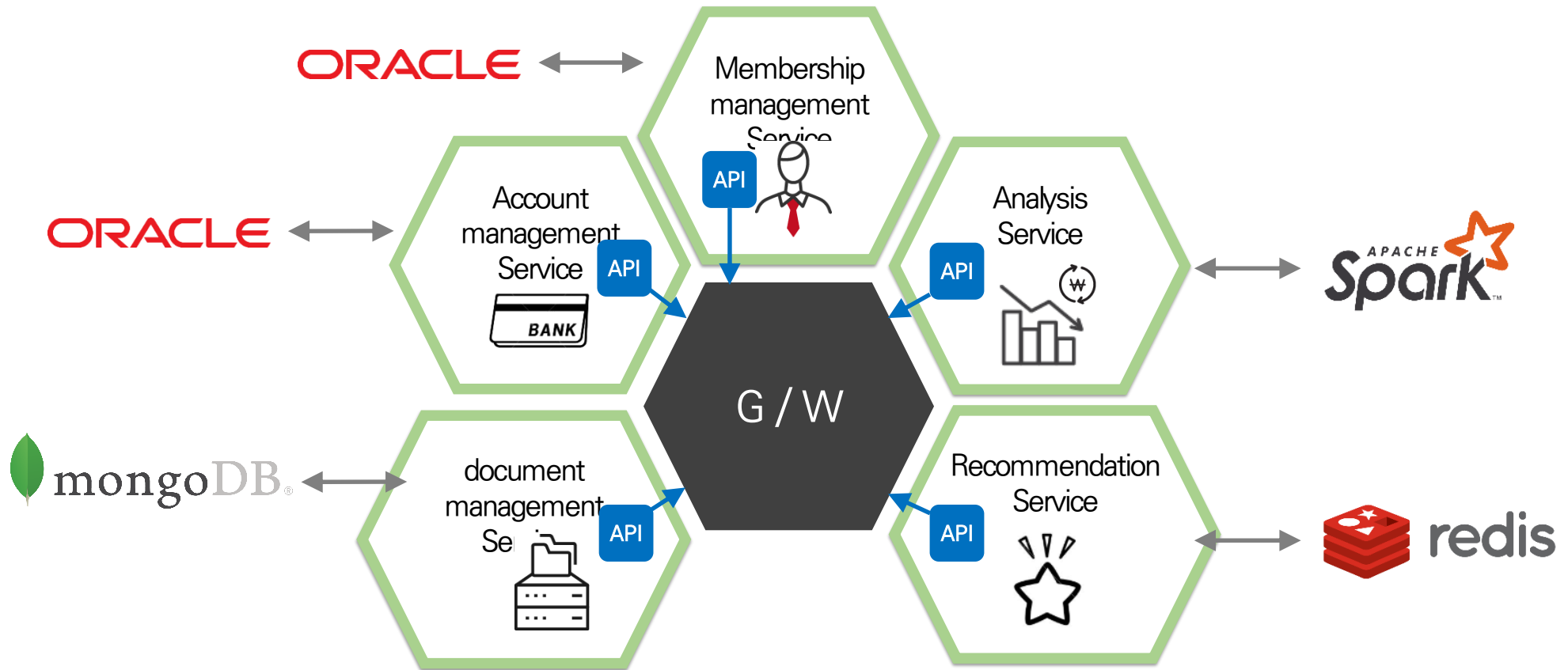
Why use Cloud Native Application?

» Emerging problems with existing applications

- ✓ Difficulty introducing new technologies
 - Non-flexible when trying to introduce different technologies for functional/non-functional characteristics per component
 - e.g.) For components with high IO operations, such as file upload/download, node.js may be recommended, but if the application is developed in Java, it is very difficult to add other technologies
 - On-Premise Cloud, CI-associated deployment automation (Jarvis) and future difficulties in connecting with Container technologies such as Docker
- ✓ Rigidity
 - Disconnecting systems, difficulty in separating DBs
 - Difficult to respond flexibly to increased inter-system connectivity
 - Difficult or impossible to introduce new versions or technologies
 - Act as a resistance or barrier to changes caused by enlargement of organization, code enlargement

Why use Cloud Native Application?

» Application of a variety of technologies

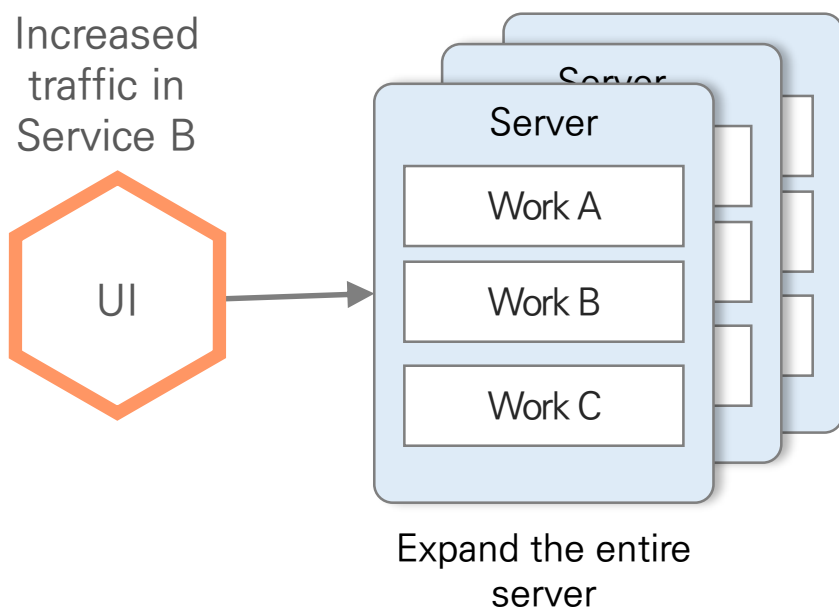


Why use Cloud Native Application?

» Optional Extensions

traditional architecture

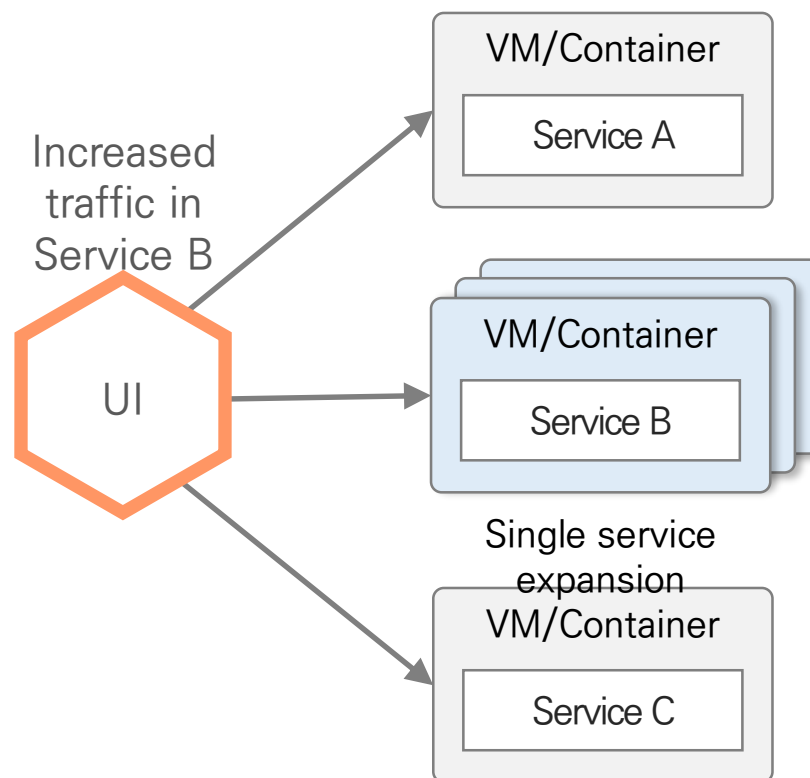
- ✓ Scale to a full server unit even as traffic increases for a particular task



VS

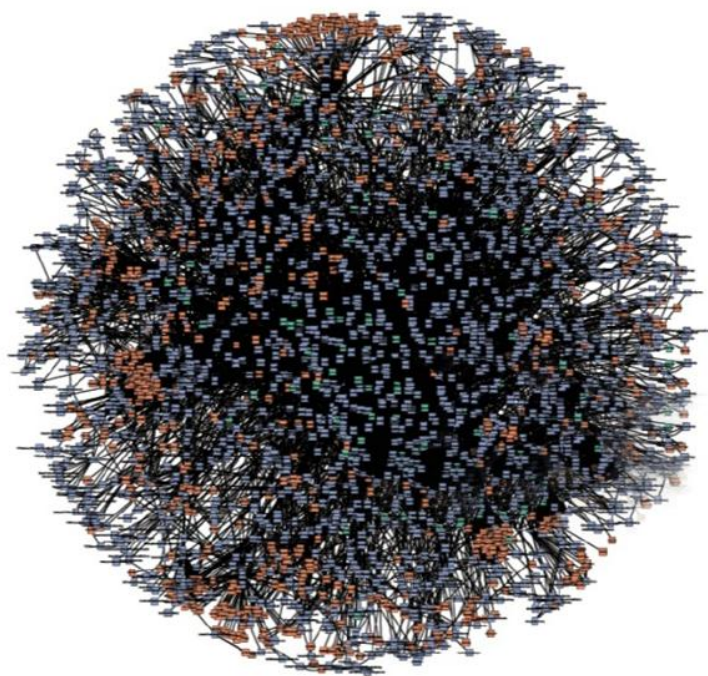
Cloud Native Architecture

- ✓ Flexibility to expand and reduce only containers of increasing traffic



Why use Cloud Native Application?

» Need for lightweight, easy deployment



Thousands of teams (Autonomous DevOps Team)

× Microservice Architecture

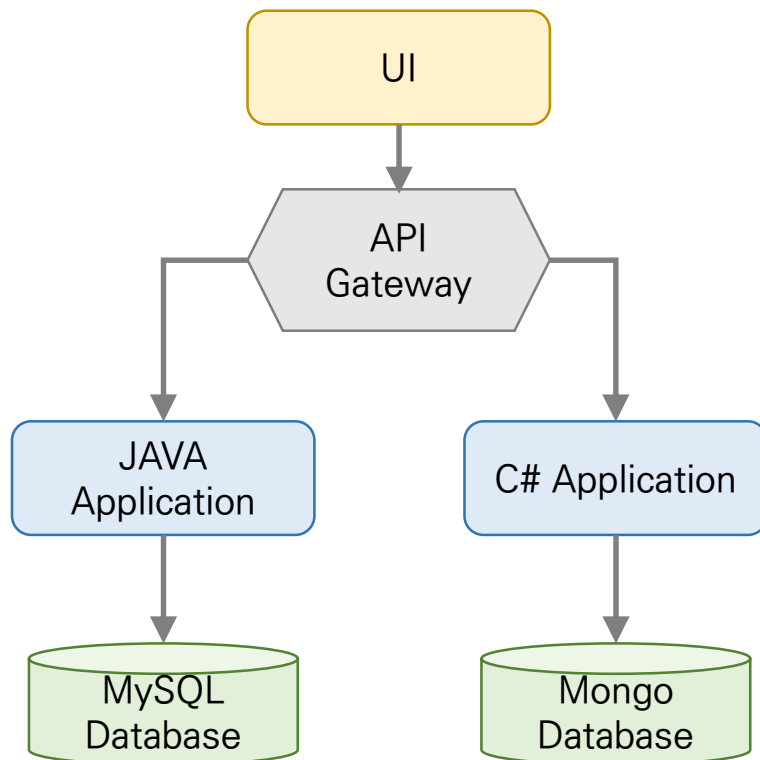
× Continuous Deployment (CD)

× Various development environments

= Distribution 50 million times per year (2014)
(5708 times per hour, or 0.63 seconds)

» Advantages of cloud native applications

Advantages (1/5)

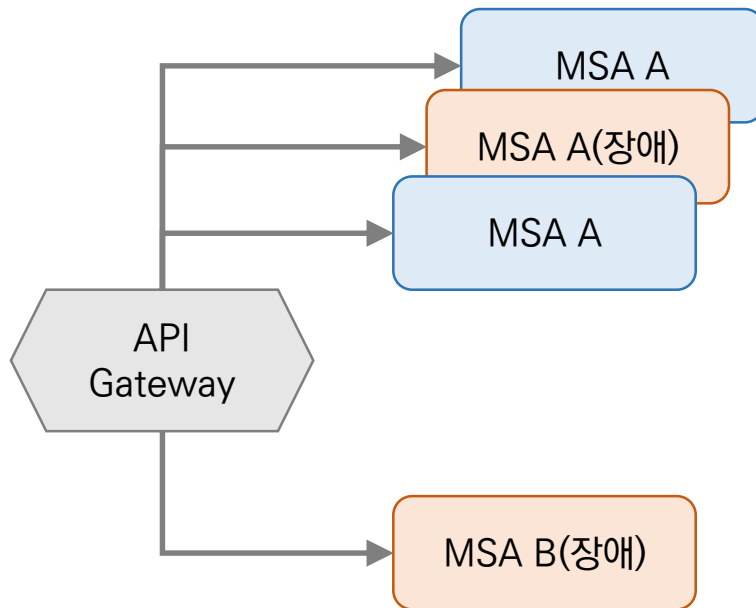


Choose optimized language and architecture

- ✓ Each Micro Service that makes up the Cloud Native Application operates independently
- ✓ Calls between services are loosely coupled via HTTP Restful API
- ✓ Can communicate according to API conventions even if each system has a different programming language or uses different databases
- ✓ Configure language and architecture optimized for each service

» Advantages of cloud native applications

Advantages (2/5)

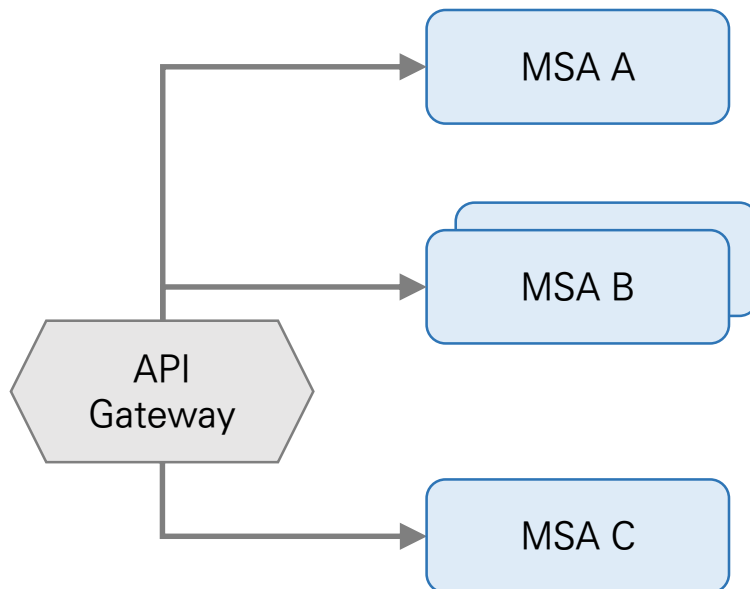


Flexible system operation

- ✓ Avoid meaningless connections by excluding them from the Request feature service until they are recovered in the event of a failure
- ✓ Horizontal Expanded Services operate normally in the event of a failure, excluding instances that fail through the load balancer

» Advantages of cloud native applications

Advantages (3/5)

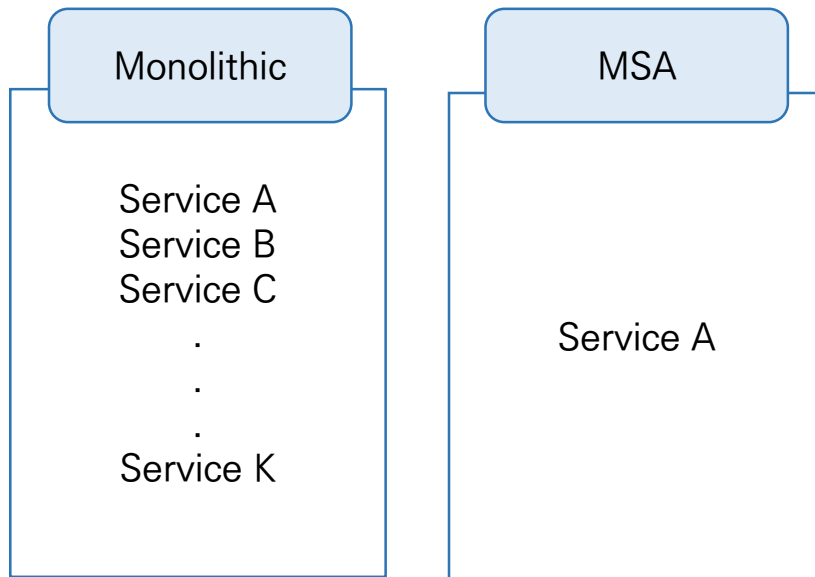


Independent scalability

- ✓ Services are independent of each other and can be scaled to service units without affecting other services
- ✓ Perform multi-application expansion, called X-axis expansion, and Z-axis expansion (Partitioning or Sharding) independently
- ✓ Scale systems with fewer resources compared to monolithic methods that require scaling the entire system

» Advantages of cloud native applications

Advantages (4/5)

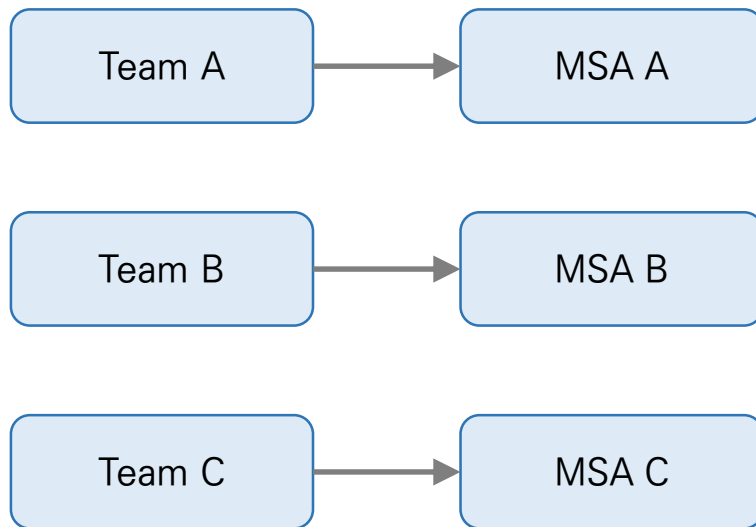


easy development

- ✓ Each microservice is a simple, single-function structure that is highly efficient for speed and improvement in development
- ✓ Fast deployment with cloud hardware and software
- ✓ Automated unit and scenario testing to maintain high quality despite fast deployment cycles

» Advantages of cloud native applications

Advantages (5/5)

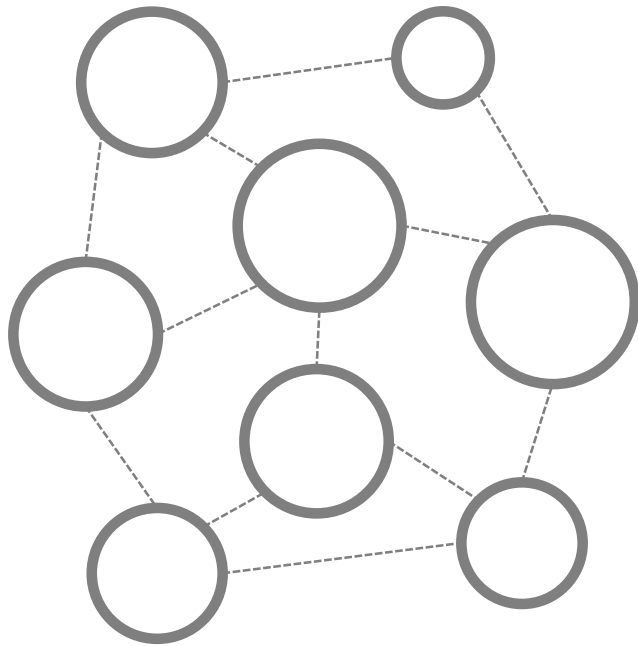


Deployment via scrum configuration

- ✓ Each microservice can be developed and distributed independently by individual teams
- ✓ Manage additional overhead as systems grow in size to a certain level

» Cloud Native Application Disadvantage

disadvantage(1/3)



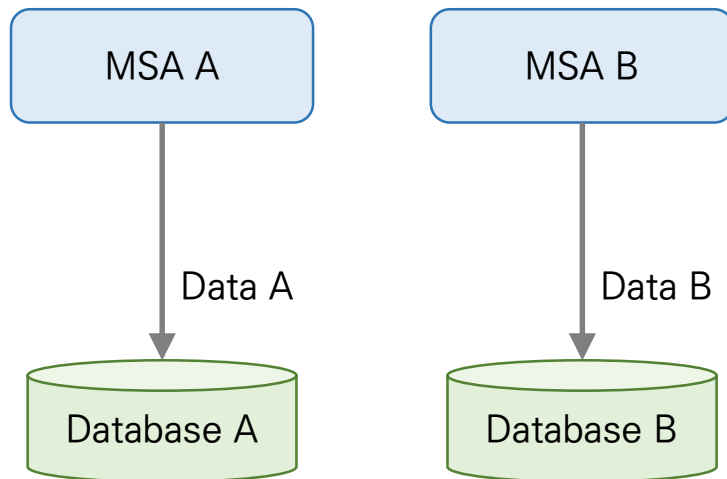
Increasing complexity

- ✓ Each service in the system is microservices, increasing management points
- ✓ Increasing architectural complexity for the entire system makes it difficult to understand the overall structure
- ✓ Increasing factors to consider when adding distributed systems, such as network latency, fault tolerance, and serialization

» Cloud Native Application Disadvantage

disadvantage(2/3)

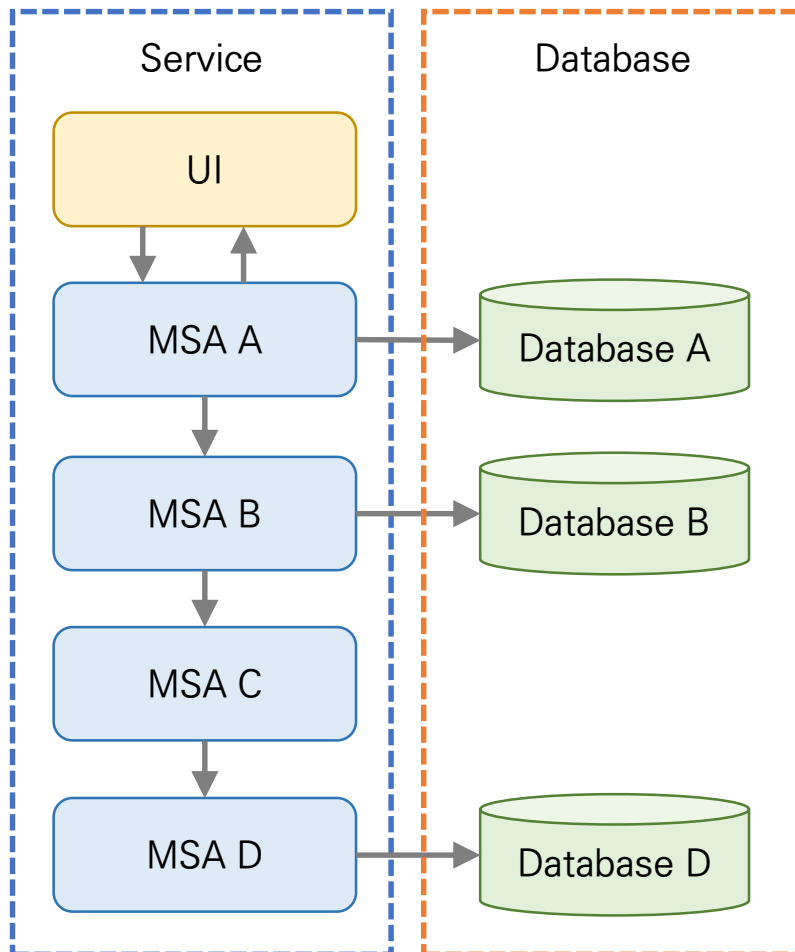
Increasing data redundancy



- ✓ Distributed databases require redundant data storage (semi-normalization)
- ✓ Each database stores its own data, increasing the likelihood of data integrity being compromised
- ✓ Multi-databases may require transaction processing, increasing the likelihood of difficulty managing transactions

» Cloud Native Application Disadvantage

disadvantage(3/3)



Difficulty in end-to-end testing

- ✓ Due to the structure of microservices, multiple microservices can be linked for single business logic processing
- ✓ It is necessary to track all MSA nodes used in the event of an exception to check the error case.
- ✓ Difficulty in tracking errors due to different development and operations personnel of each service

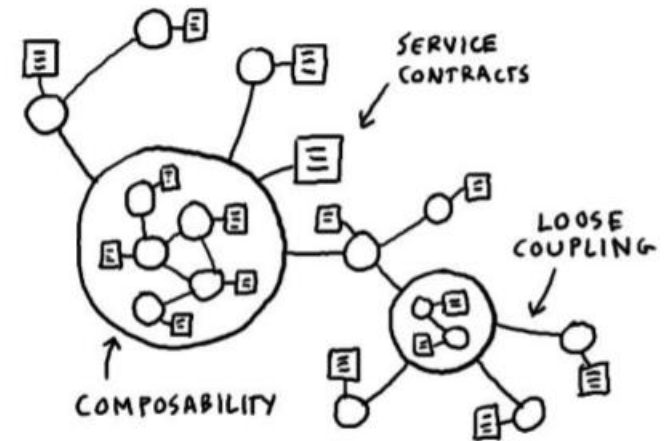
A top-down view of a wooden desk with a laptop, glasses, a cup of coffee, and a small plant. The text "03. MSA(Micro Service Architecture)" is overlaid on the left side of the image.

03. MSA(Micro Service Architecture)

» MSA Overview

MSA Overview

- ✓ **One Thing**
Services focused on performing one function (business-related functions and roles)
- ✓ **Small**
Separate to the smallest unit of service that is independent and deployable
- ✓ **API**
Associate with other services via API
- ✓ **Autonomous**
Develop and operate independently; an independent team is responsible for the development and operation of each service



"Micro-service architecture style means a single application

Run as its own process and communicate with lightweight mechanisms (mainly HTTP resource APIs)

It's a way of developing it as a collection of small services."

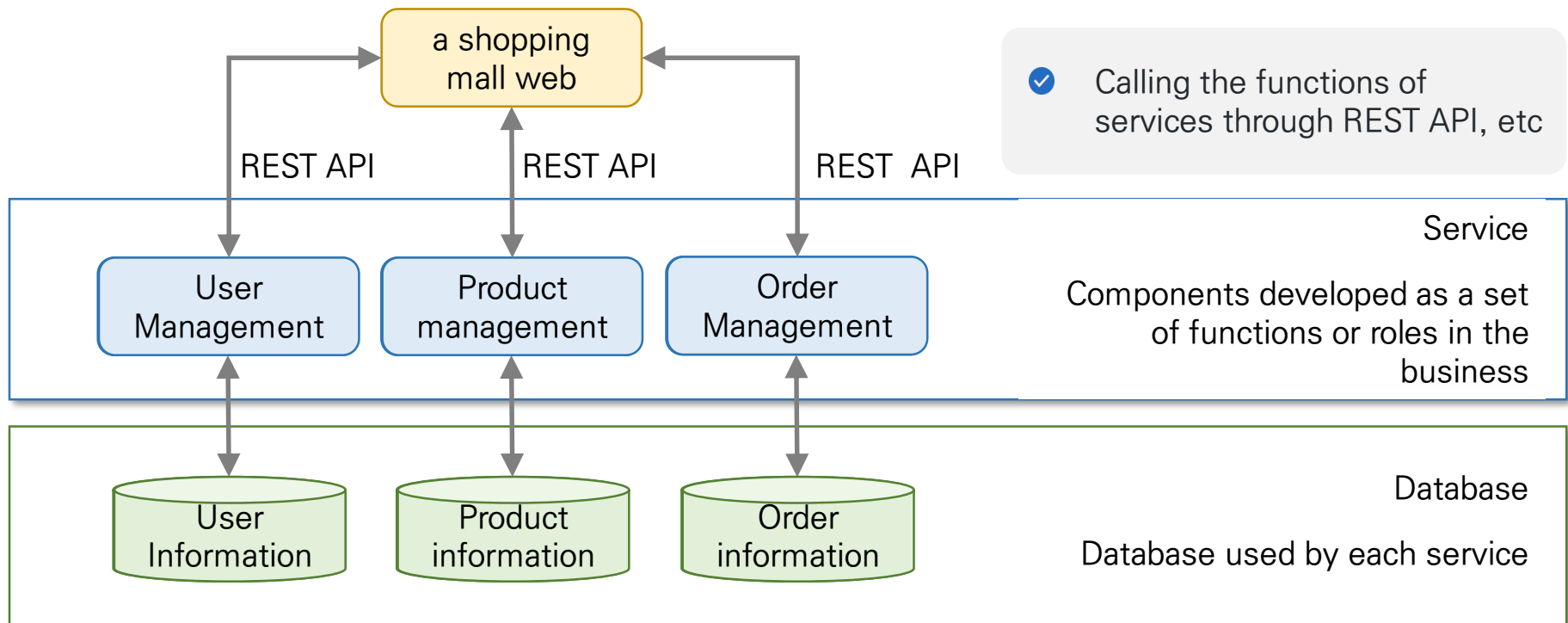
– Martin Fowler

01 MicroService Architecture

» Examples of Micro-Service Architecture

Examples of Micro-Service Architecture

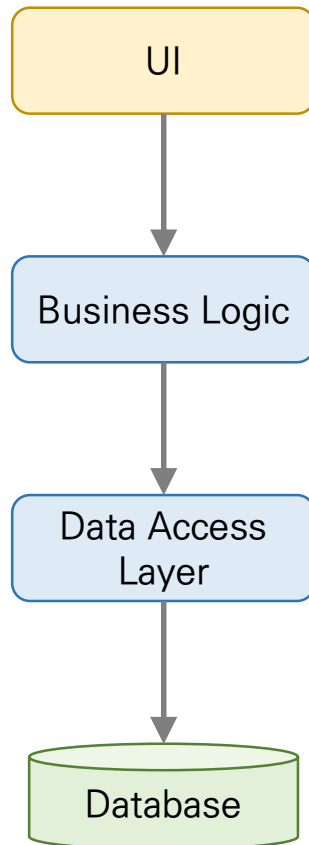
- ✓ Provides functionality by combining multiple services to implement one function
- ✓ e.g.) Ordering: User information inquiry, product information inquiry, new order creation



01 MicroService Architecture

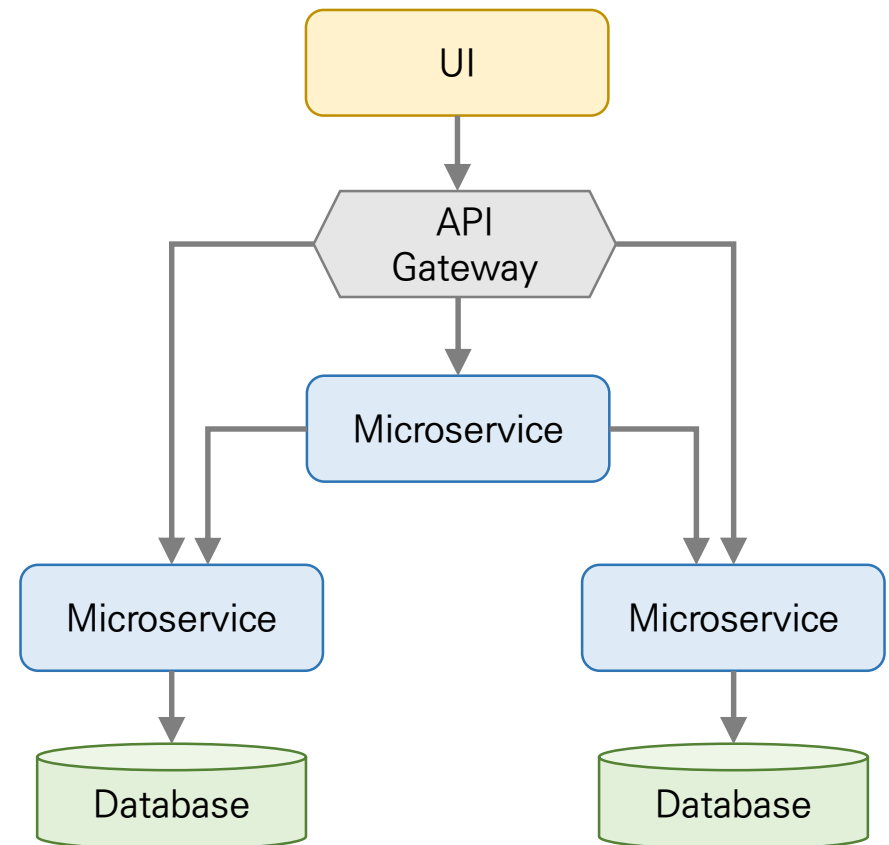
» Monolithic VS MicroService

Monolithic



VS

MicroService



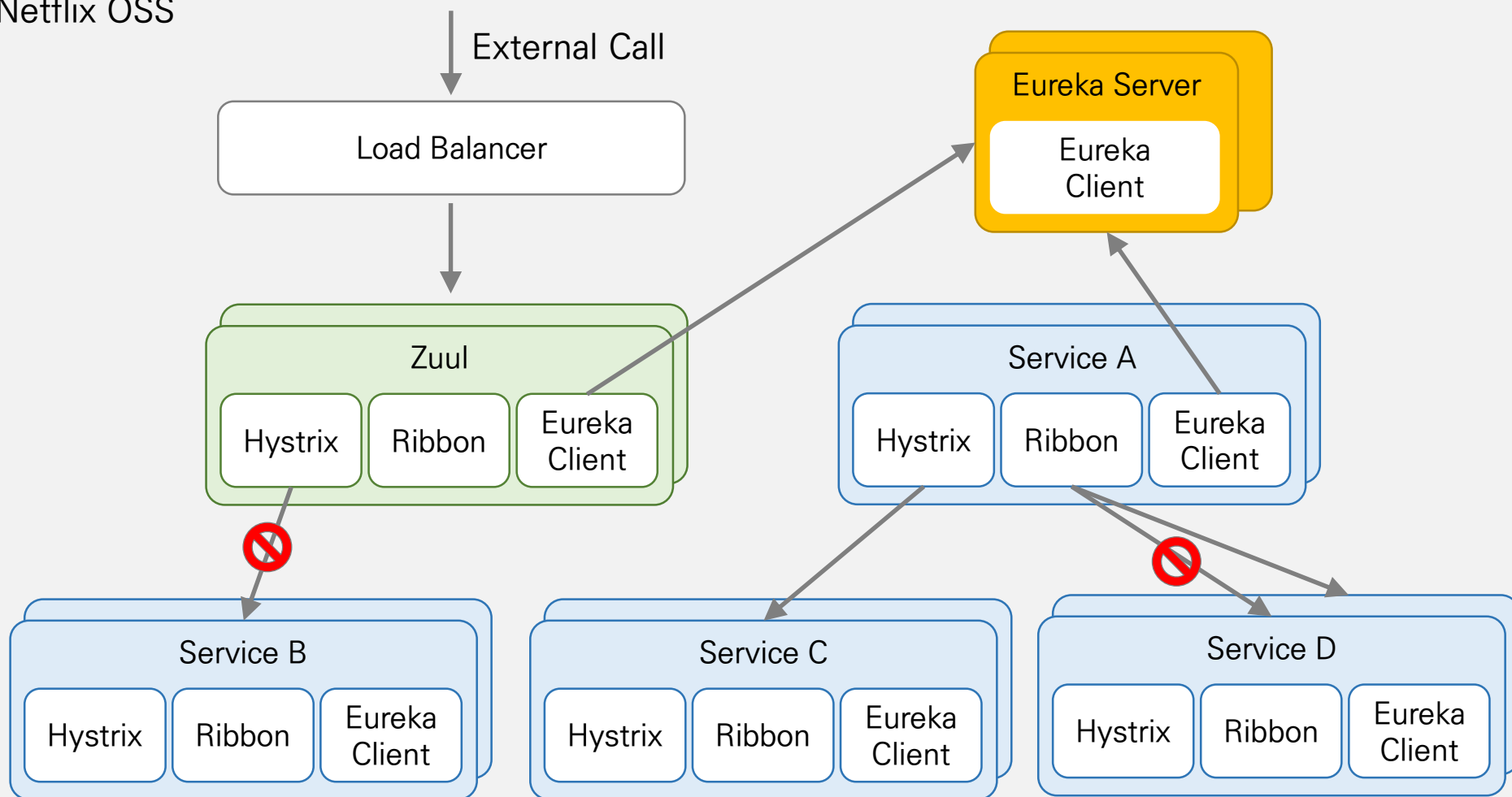
» What is Netflix OSS?



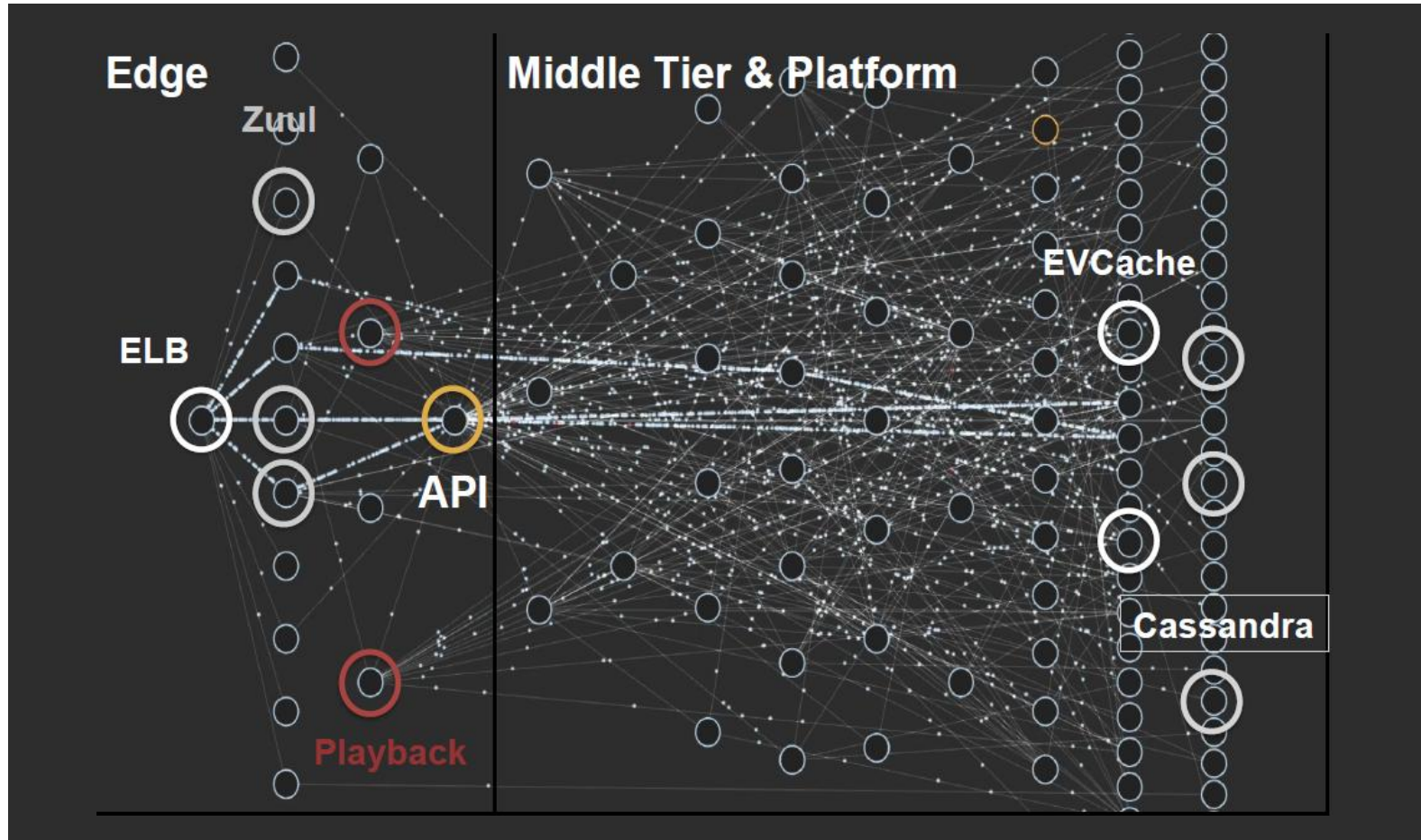
- ✓ Netflix OSS stands for Netflix Open Source Software, which shares MSA transformation technology know-how and troubleshooting methods for moving to reliable, horizontally scalable cloud systems since Netflix previously experienced severe server failures
- ✓ Representative open sources include Eureka, Hystrix, Ribbon, and Zuul.

» Netflix MSA Architecture

Netflix OSS

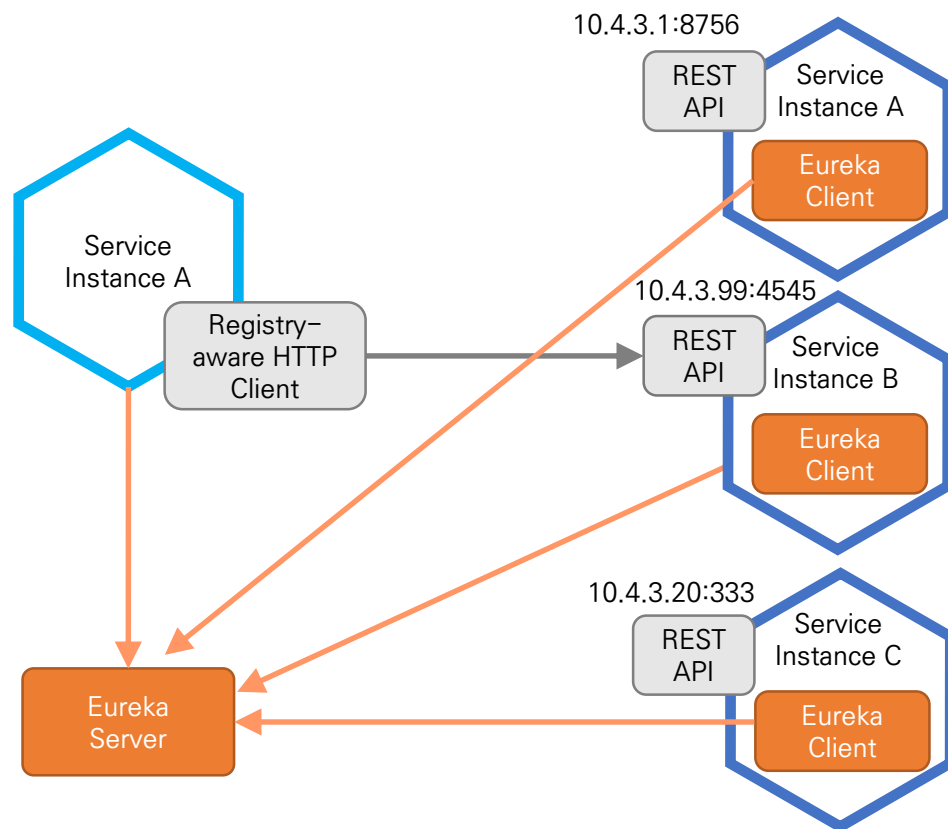


» How Netflix MSA system calls appear



» Eureka

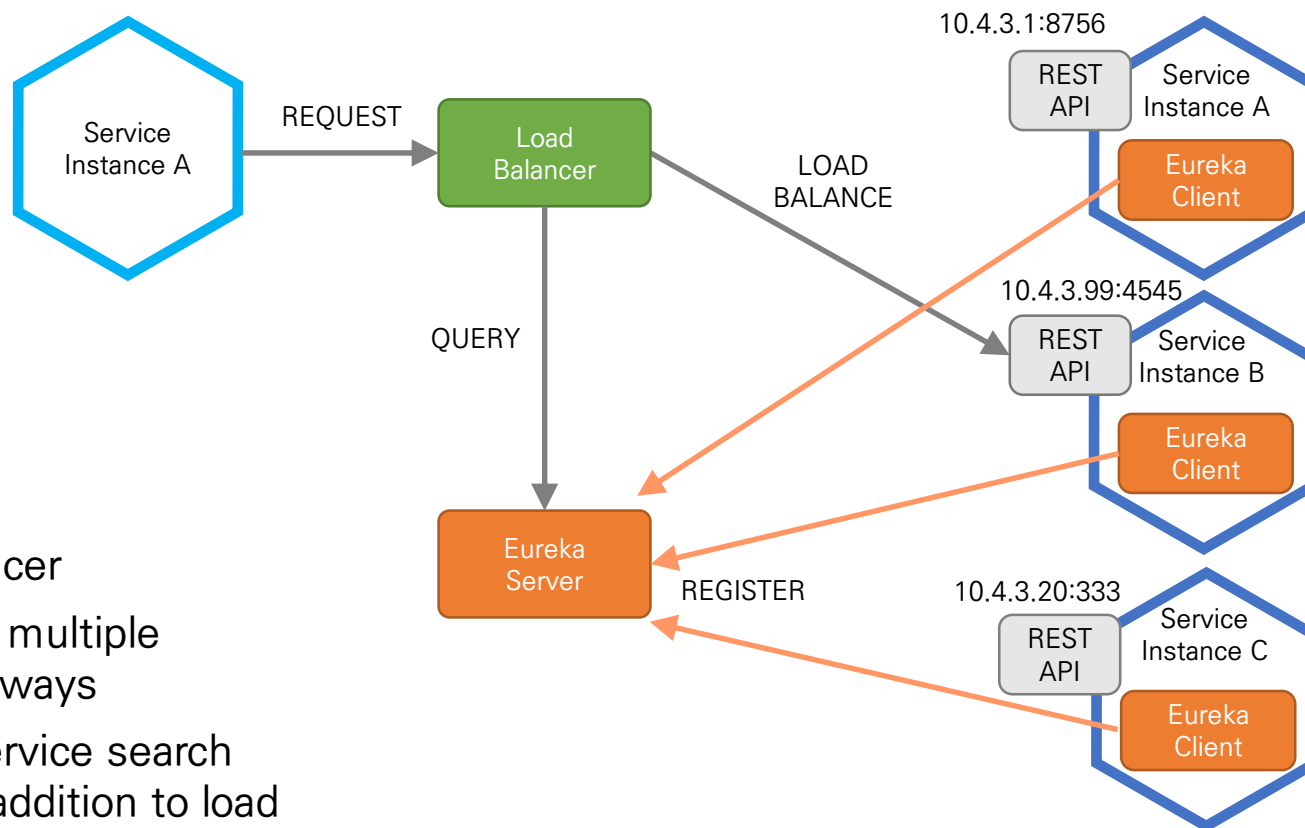
Eureka



- ✓ Services that provide and manage service creation information so that dynamically changing service instances can be called as Service Discovery & Registry
- ✓ Component Eureka Server and Eureka Client
- ✓ Eureka Client registers its meta information (IP, port, etc.) with Eureka Server
- ✓ Eureka Server receives Heartbeat from each client every 30 seconds and health checks to manage the client

» Ribbon

Ribbon



- ✓ Client Side Load Balancer
- ✓ Load balancing across multiple servers in a variety of ways
- ✓ Provides integrated service search and fault tolerance in addition to load balancing capabilities

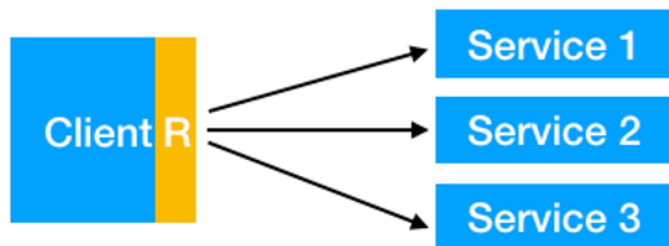
» Client Side Load Balancing vs Server Side Load Balancing

Client Side Load Balancing

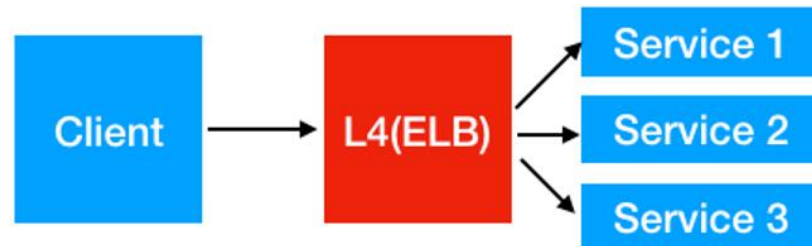
VS

Server Side Load Balancing

Client Side Load Balancing (Ribbon)



Server Side Load Balancing (Existing)



- ✓ Load balancing through client programs rather than using physical hardware equipment
- ✓ Flexible expandability
- ✓ Automatically add and delete server lists

출처 : <https://sabarada.tistory.com>

- ✓ How to load balance physical hardware (L4 Switch)
- ✓ Using physical equipment is costly and reduces flexibility
- ✓ Manually add a list of servers

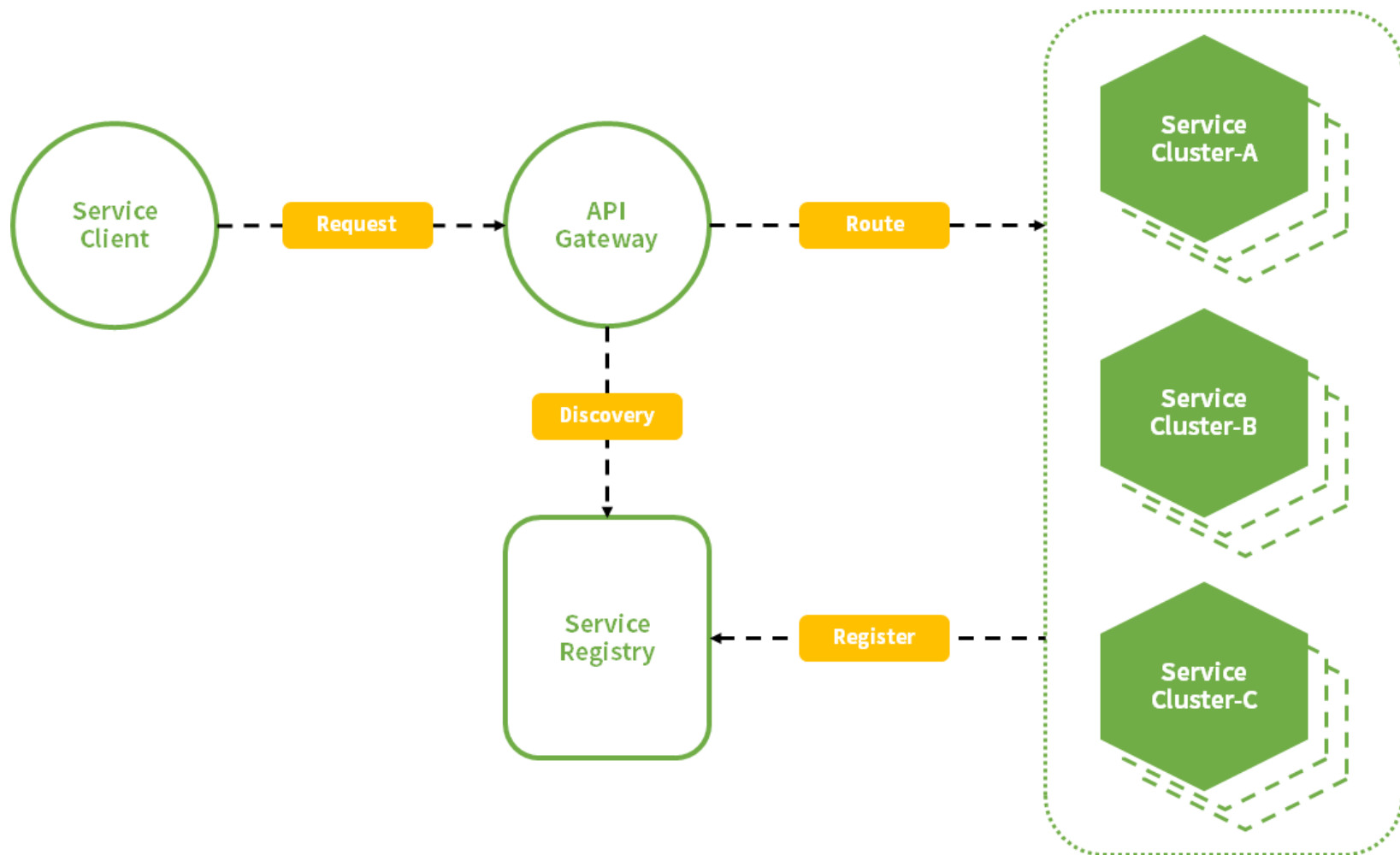
» Zuul

Zuul



- ✓ API Gateway from Netflix OSS
- ✓ Netflix OSS provides a single endpoint for API calls via Zuul
- ✓ Zuul is an L7 application gateway that provides features for dynamic routing, monitoring, resilience, security, and more
- ✓ In addition to simple routing, it also provides filtering to handle HTTP messages by dynamically intercepting requests and responses

» Zuul Architecture



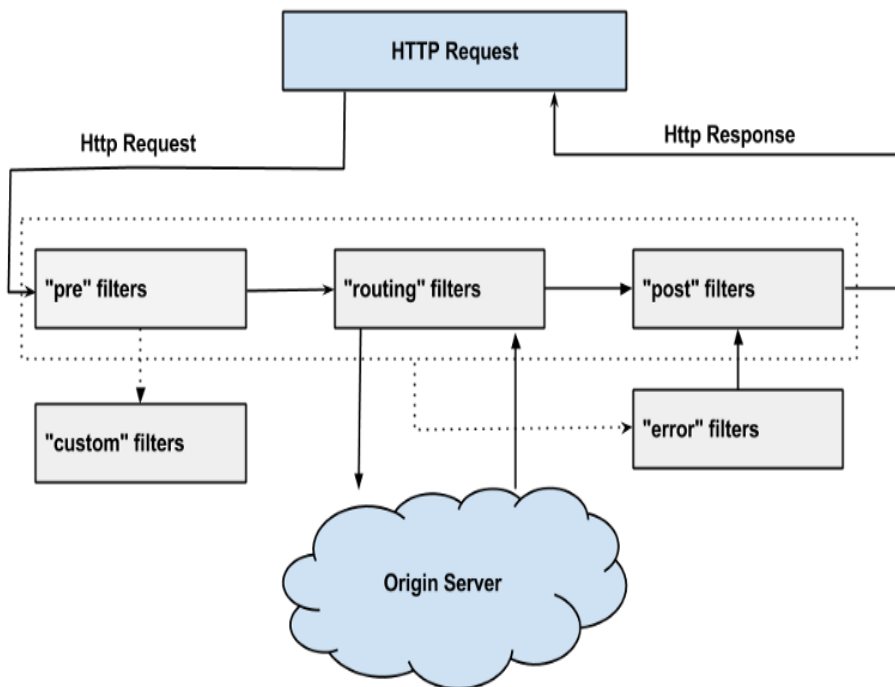
» Zuul Features

Zuul Features

- ✓ Authentication and Security: on client requests, identify authentication requirements for each resource and reject requests that do not satisfy them
- ✓ Insights and Monitoring: Provides meaningful data and statistics
- ✓ Dynamic Routing: dynamically routing requests to other clusters as needed
- ✓ Stress Testing—Gradually increases cluster traffic to measure performance
- ✓ Load Shedding : Allocates capacity for each type of request, limiting requests exceeding
- ✓ Static Response Handling: Can be configured to respond to specific requests immediately instead of sending traffic to the backend

» Zuul Filter

Filter Type



- ✓ **Pre Filter**
Runs prior to routing and is used for client authentication/authorization, etc
- ✓ **Route Filter**
Not just relying on URI patterns for routing, but using more complex mechanisms
- ✓ **Post Filter**
Runs after routing and takes advantage of frequent manipulation of response messages, such as status codes
- ✓ **Error Filter**
Run when an error occurs in the filter

» REST API Definition

REST API

{ REST API }

Representational State Transfer API

- ✓ Representative State Transfer is an abbreviation that refers to the exchange of the status of a resource by name
- ✓ APIs designed in compliance with REST APIs are referred to as RESTful APIs

» Features of REST API

Features of REST API

- ✓ Uniform Interface
 - HTTP standards alone enable resource manipulation with available URIs without being dependent on specific platforms (iOS, WEB, Android), languages, and technologies
- ✓ Client – Server
 - REST servers provide APIs to directly manage features such as client authentication, making client and server content clearer and less dependent on each other
- ✓ Statelessness
 - No separate management of health information increases freedom of service and simplifies implementation
- ✓ Cacheable
 - Using the HTTP web standard allows you to leverage your existing infrastructure to apply HTTP's caching capabilities
- ✓ Self-Descriptive
 - Self-expression structure is easy to understand just by looking at REST API messages
- ✓ Layered System
 - REST servers can be configured in multiple layers to add security, load balancing, encryption layers, and more

» REST API Method

REST API Method

- ✓ An architectural approach to developing an application as a small set of services communicating over an API

“REST”

GET

/user

GET

/user/:userID

POST

/user

PUT

/user/:userID

DELETE

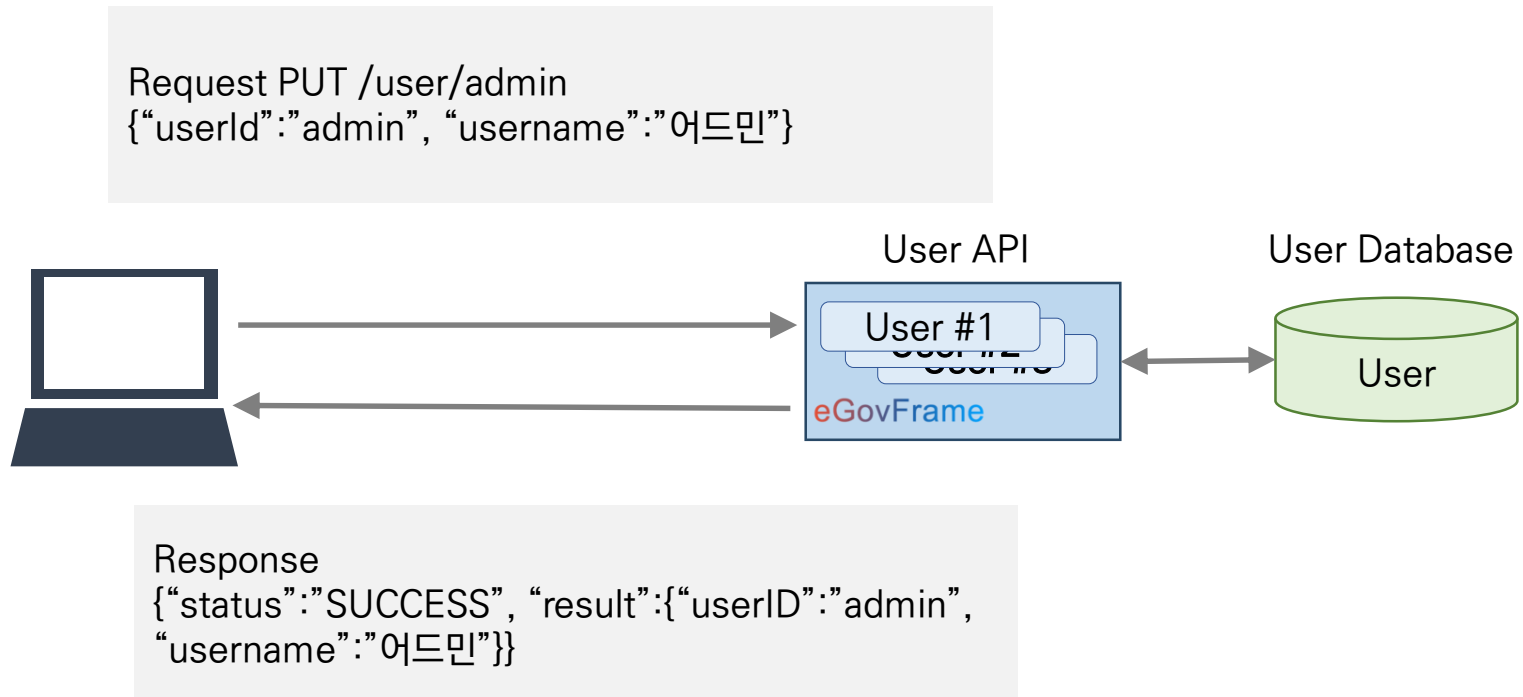
/user/:userID

Method	Function
GET	Inquiry of specific data or lists
POST	Create new data
PUT	Modify existing data
DELETE	Delete existing data

» REST API Call Example

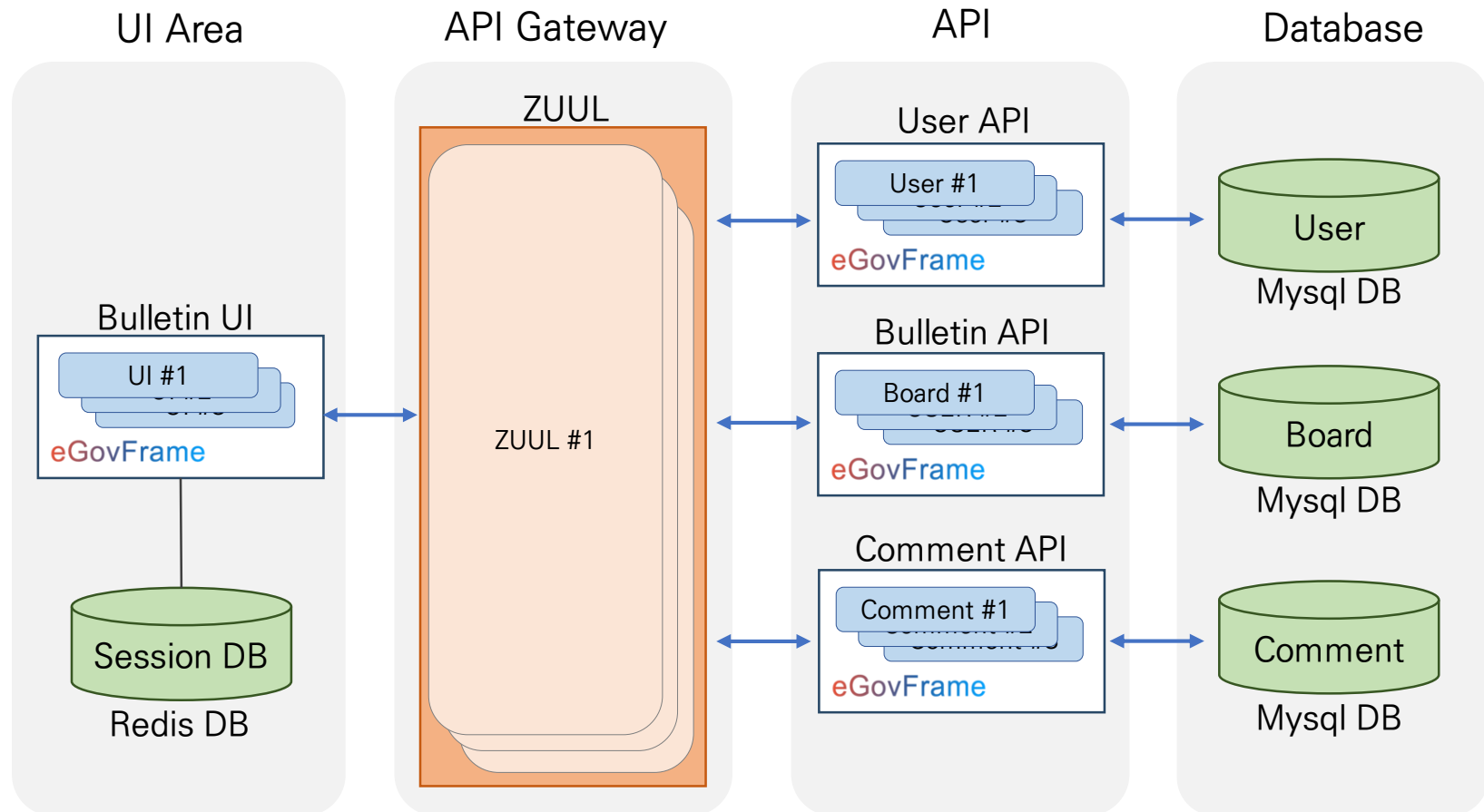
REST API Call Example

- ✓ The resources and tasks to be controlled are specified using URI and HTTP methods, and Request Parameters and Response for the task send and receive data to and from the Body area through JSON messages.



» Lab Application Configuration Diagram

Bulletin board with MSA structure (1 of 2)



» Lab Application Configuration Diagram

Bulletin of MSA structure (2 of 2)

App and Service Name	Type	Instance	memory(GB)	etc
EDU-MSA-UI	Application	2	2	UI (redundant for session synchronization processing)
EDU-MSA-ZUUL	Application	1	1	API Gateway (zuul)
EDU-MSA-BOARD	Application	1	1	Micro Service
EDU-MSA-COMMENT	Application	1	1	Micro Service
EDU-MSA-USER	Application	1	1	Micro Service
EDU-DB-BOARD	Service	1	–	Mysql Database
EDU-DB-COMMENT	Service	1	–	Mysql Database
EDU-DB-USER	Service	1	–	Mysql Database
SESSION-DB	Service	1	–	Redis Database (Session Synchronization DB)

02. Functional definition

» Define MSA-specific features

EDU-MSA-BOARD

End point	Method	TYPE	Explanation
/board	Get	R	Look up the list of posts
/board/:seq	Get	R	Single Post Query
/board	Post	C	Creating a post
/board/:seq	Put	U	Modifying posts
/board/:seq	Delete	D	Deleting a post

» Define User Table

TB_USER

Column name	Data Type	Allow null	Key	Index
user_seq	INTEGER	N	PK	
user_id	VARCHAR(20)	N		UNIQUE
user_passwd	VARCHAR(256)	N		
user_name	VARCHAR(20)	N		
use_yn	VARCHAR(1)	Y		
create_dt	DATETIME	N		
update_dt	DATETIME	N		

» Creating a User Table

TB_USER Table Creation Script

- ✓ Create TB_USER Table

```
CREATE TABLE TB_USER (  
    user_seq      INTEGER NOT NULL AUTO_INCREMENT PRIMARY KEY,  
    user_id       VARCHAR(20) NOT NULL,  
    user_passwd   VARCHAR(256) NOT NULL,  
    user_name     VARCHAR(20) NOT NULL,  
    use_yn        VARCHAR(1) DEFAULT 'Y',  
    create_dt     DATETIME NOT NULL,  
    update_dt     DATETIME NOT NULL  
);  
CREATE UNIQUE INDEX id_key ON TB_USER (user_id);
```

» GET /user

✓ Request JSON

parameter	Allow null	Explanation
searchType	Y	Search Criteria Type (ID, Name)
searchValue	Y	Search Criteria Content
page	Y	Current page (default 1)
pagePerCount	Y	List per page (default 15)

✓ Response JSON

parameter	Allow null	Explanation
userCount	N	Number of Scan Users
userList	Y	Search Users List
userSeq	N	User Sequence
userId	N	User ID
userName	N	Username
createDt	N	Date Created
updateDt	N	Modified date

» GET /user/:userID

✓ Request JSON

parameter	Allow null	Explanation
No parameters		

✓ Response JSON

parameter	Allow null	Explanation
userSeq	N	User Sequence
userId	N	UserID
userName	N	User name
createDt	N	Date Created
updateDt	N	Modified date

» POST /user

✓ Request JSON

parameter	Allow null	Explanation
userId	N	User ID
userPasswd	N	User password
userName	N	Username

✓ Response JSON

parameter	Allow null	Explanation
userSeq	N	User Sequence

» PUT /user/:userID

✓ Request JSON

parameter	Allow null	Explanation
userPasswd	N	User password
userName	N	Username

✓ Response JSON

parameter	Allow null	Explanation
No parameters		

» DELETE /user/:userID

✓ Request JSON

parameter	Allow null	Explanation
No parameters		

✓ Response JSON

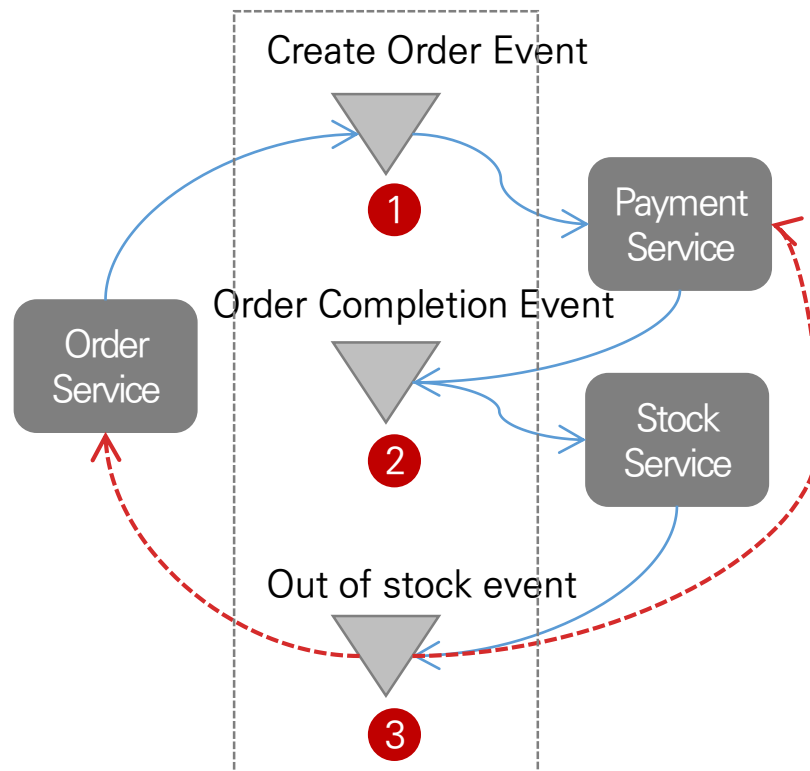
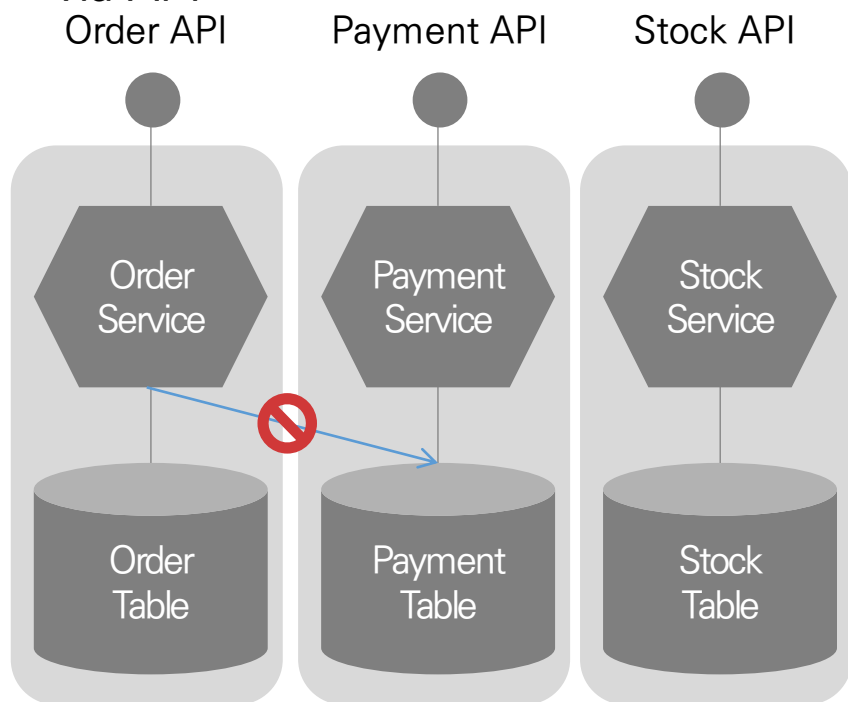
parameter	Allow null	Explanation
No parameters		

» Worried about DB transaction processing

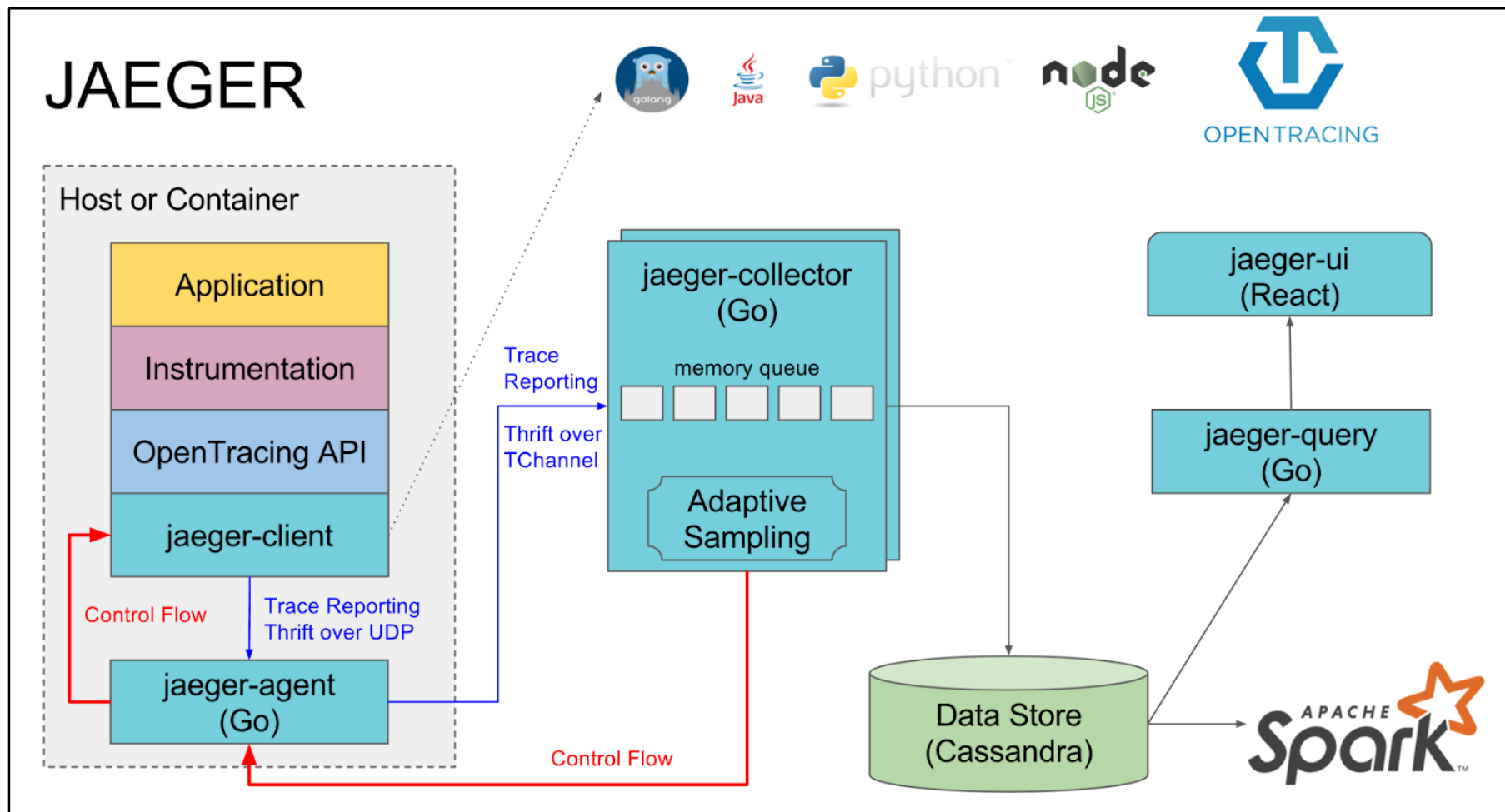
Handling Data Consistency Between Microservices

- ✓ Transaction Processing by Microservice
- ✓ Accessing data from other services via API

- ✓ Event-Driven Architecture
- ✓ Compensation transaction handling in case of transaction failure



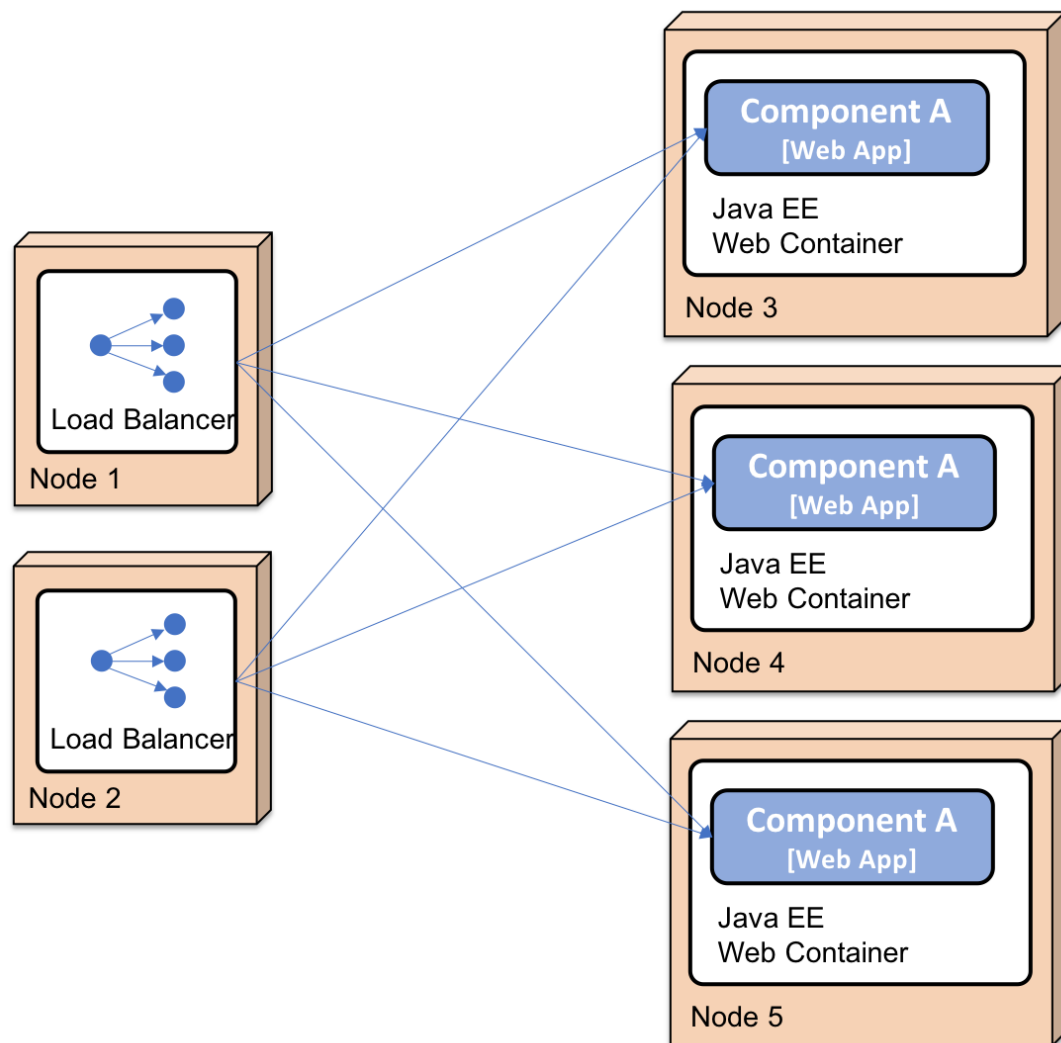
» Establish a policy for collecting distributed logs



What are cloud native applications?

» Microservice Design Patterns

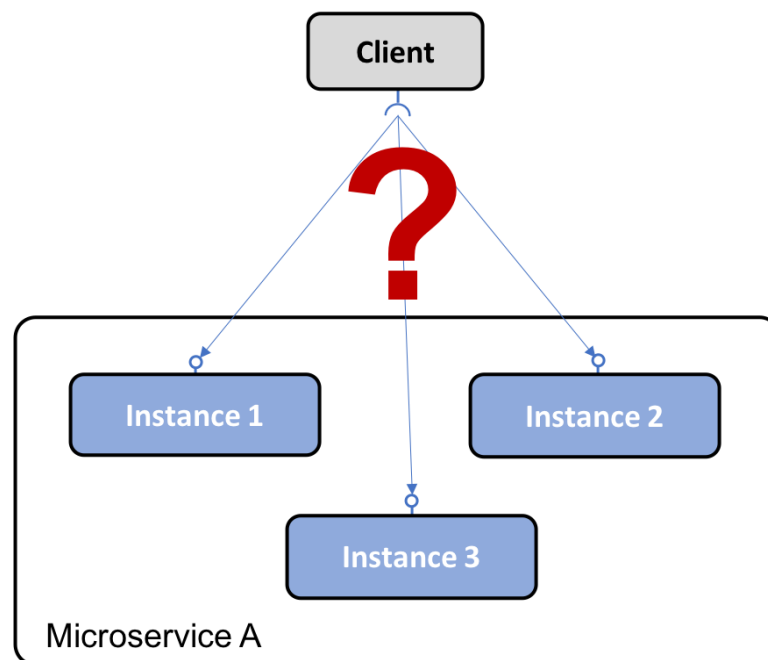
- ✓ Search for services
- ✓ Edge Server
- ✓ Reactive Microservices
- ✓ Centralize configuration
- ✓ Centralize Log Analysis
- ✓ Distributed Tracking
- ✓ circuit breaker
- ✓ Control Loop
- ✓ Centralize monitoring and alerts



What are cloud native applications?

» Search for services

- ✓ A problem
 - Microservice instances running on containers are assigned dynamic IP addresses at startup
 - Makes HTTP-based REST APIs that microservices expose difficult for clients to call
- ✓ Solution
 - Automatically register and terminate microservices and instances
 - Clients must be able to send requests to logical endpoints of microservices
 - Requests for microservices must be load balanced to available instances
 - Abnormal instance detection must be possible



What are cloud native applications?

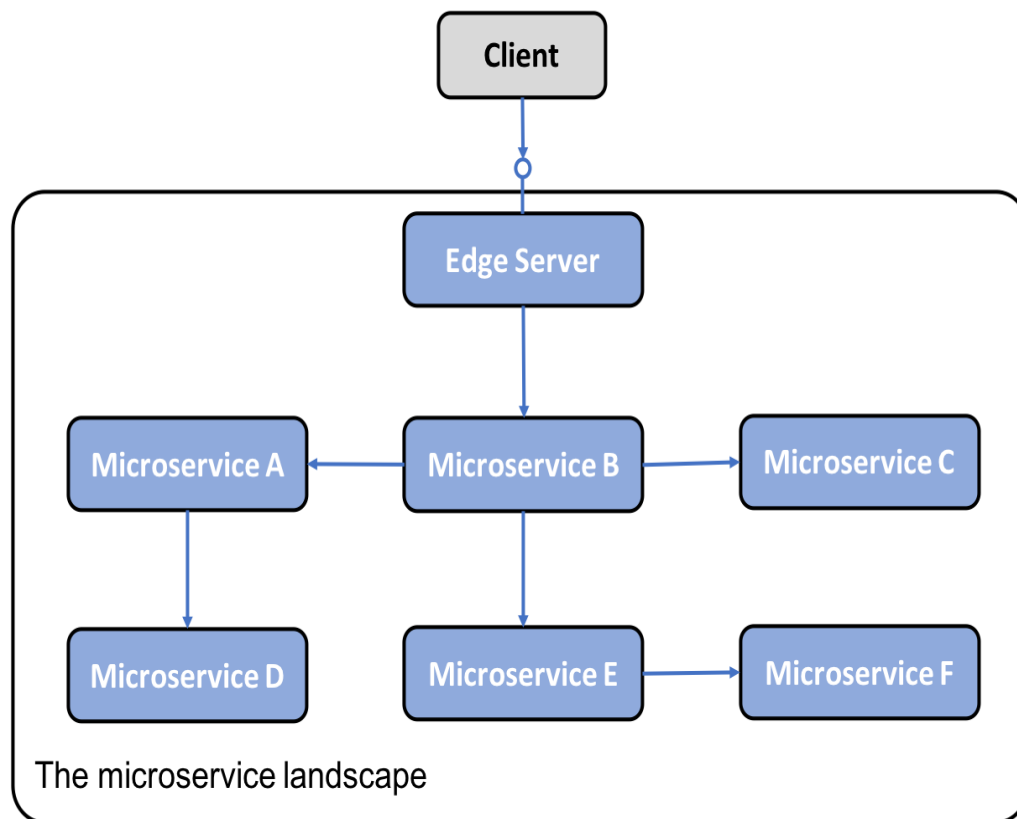
» Edge Server

✓ A problem

- Only some of the microservices should be disclosed to the outside world and other microservices should be hidden from the outside world
- Must be protected from malicious client requests

✓ Solution

- Added a new component (edge server) through which all requests pass
- Check client reliability using standard protocols and OAuth, OIDC, JWT tokens, API keys, etc



» Reactive Microservices

- ✓ a problem
 - When using blocking I/O such as HTTP-based Rest JSON API, the operating system thread is occupied, and when the number of requests increases, the response is slowed down due to lack of available threads and server interruptions occur

- ✓ Solution
 - Use non-blocking I/O to prevent threads from being assigned while waiting for the database or other microservices to process
 - Use asynchronous models or a reactive framework
 - Need to be designed to be resilient to respond to any disruption in service

» Centralize configuration

- ✓ a problem
 - Need a quick view of all running microservice instances' configuration information
 - How do I update the configuration and have all relevant microservice instances updated in batches?

- ✓ Solution
 - Add components that store configuration information for all microservices

What are cloud native applications?

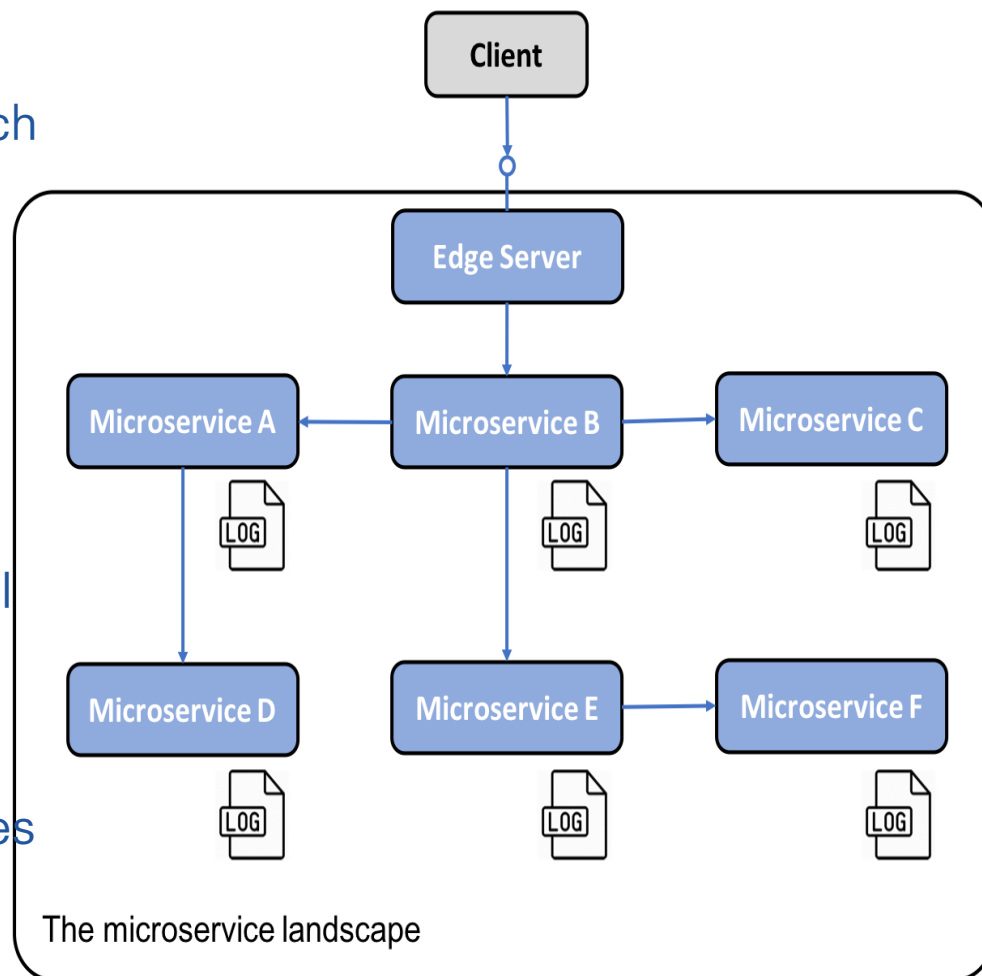
» Centralize Log Analysis

✓ a problem

- How do I get an overview of what happens to the entire system when each instance of microservice logs a log file locally?
- How do I find an instance of microservice where the problem occurred and write an error message in the log file?
- If an end user finds a problem, how do I find a log message related to it?

✓ Solution

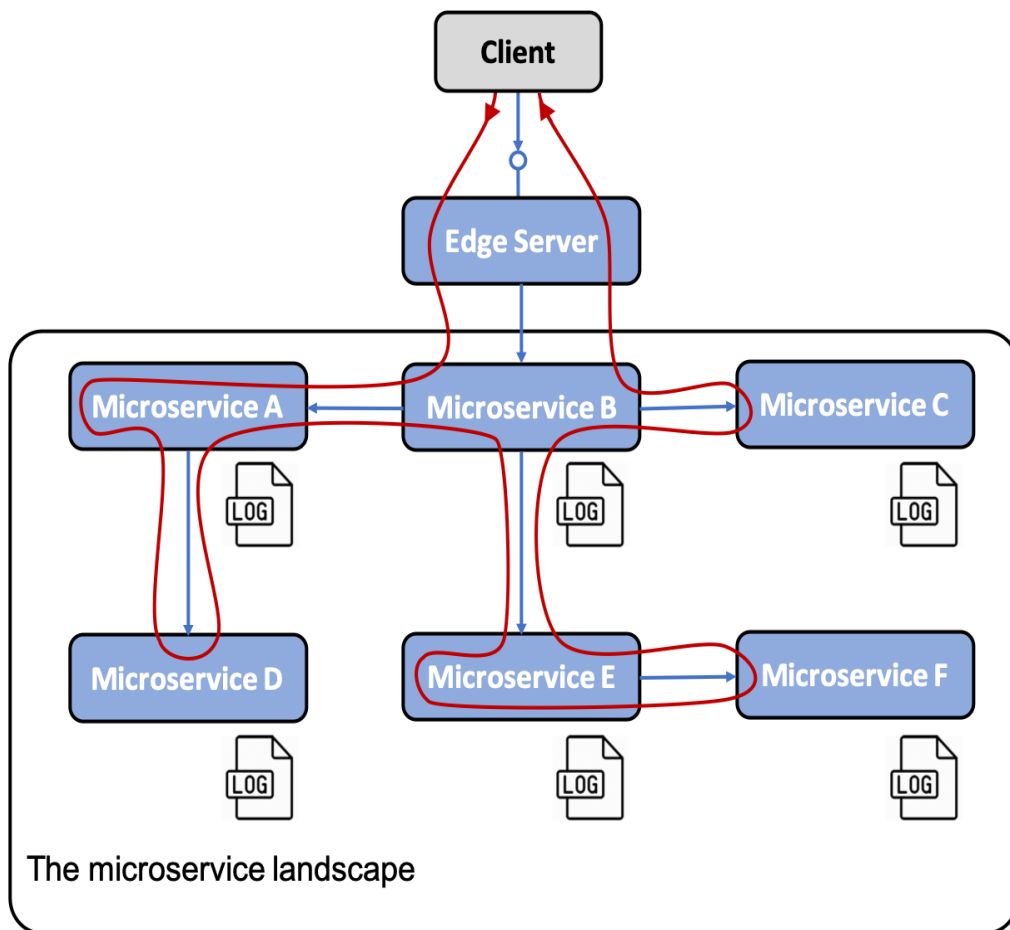
- Collect log events by detecting instances of microservice
- Store log events in a structured and searchable format



What are cloud native applications?

» Distributed Tracking

- ✓ a problem
 - To find the microservices that caused the problem in the event of a failure and identify the root cause?
 - To find log messages for all microservices associated with relaxation in the event of a failure?
- ✓ Solution
 - Correlation ID must be placed in relevant requests and messages and Correlation ID must be present in all log events



What are cloud native applications?

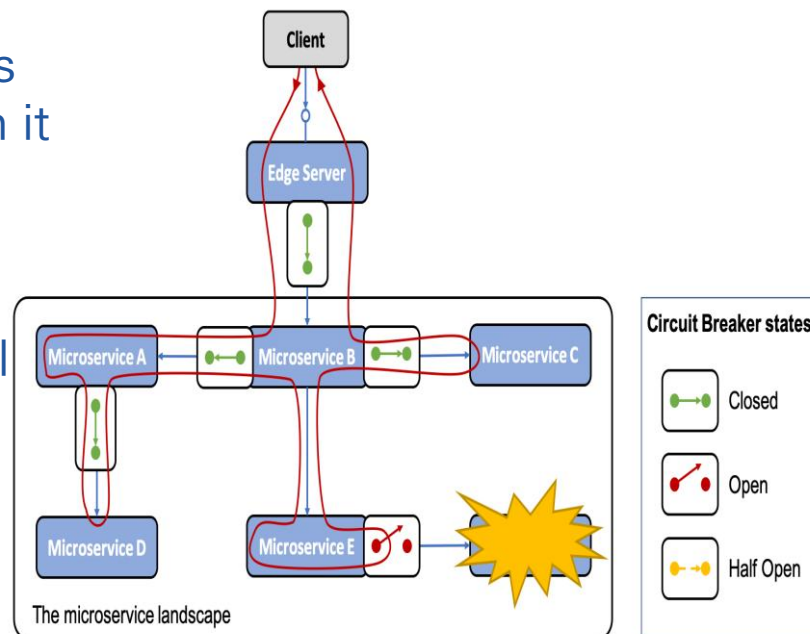
» circuit breaker

✓ a problem

- Synchronous communication microservices can cause chain failures

✓ Solution

- Add circuit breakers that block new requests from being sent to a particular service when it is in trouble
- Open circuit when detecting service issues
- Use half-open circuit. Check service normal operation
- Circuit closed when normal operation is confirmed



What are cloud native applications?

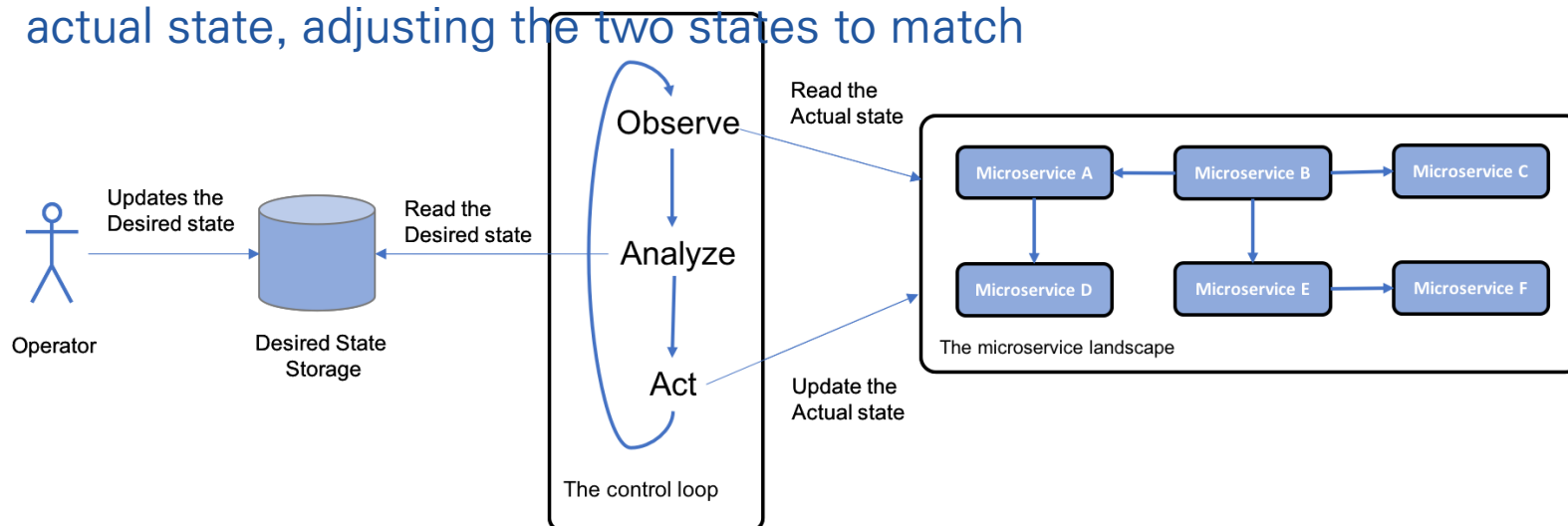
» Control Loop

✓ a problem

- Difficult to manually detect and cope with instances of microservices in environments where multiple microservices are distributed across multiple servers

✓ Solution

- Add new components (control routes) to the system environment to observe the state of the system environment
- This component continuously observes the operator-specified state and the actual state, adjusting the two states to match



What are cloud native applications?

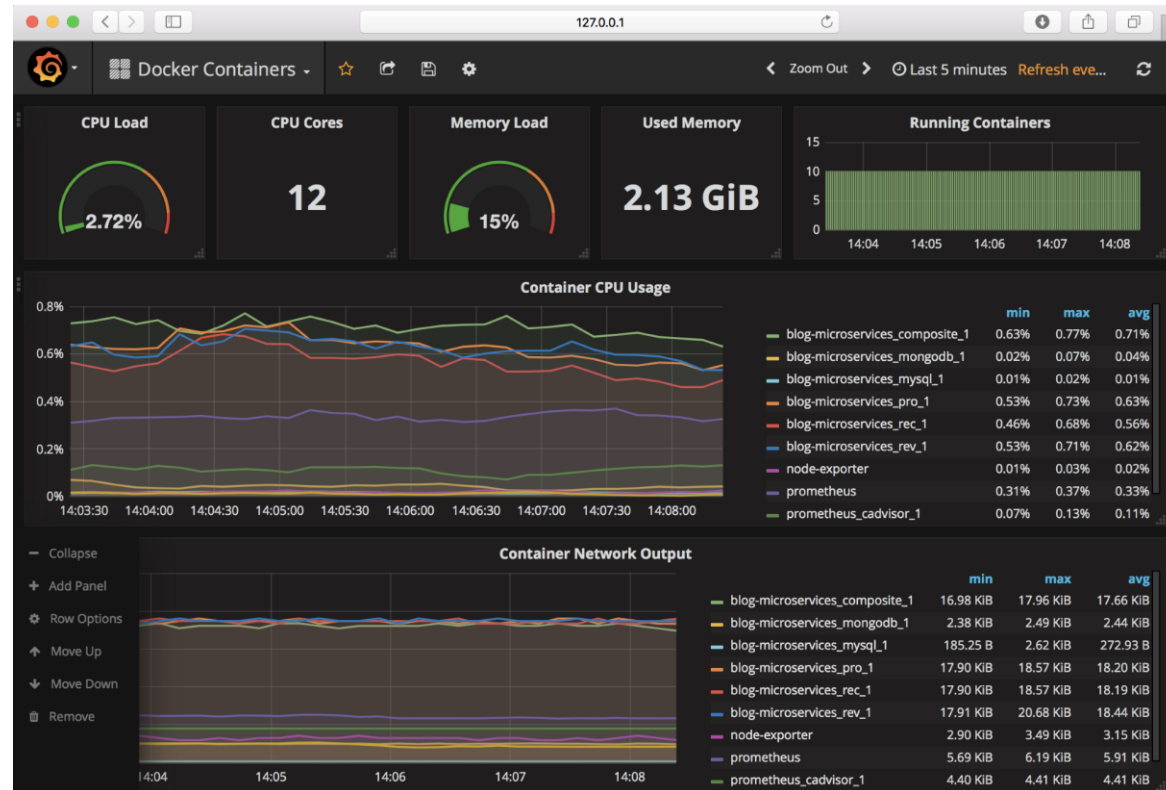
» Centralize monitoring and alerts

✓ a problem

- Finding the root cause of the problem is difficult when response times or H/W resource usage is too high.
- Enable analysis of H/W resource usage by microservice

✓ Solution

- Add components that collect metrics for H/W resource usage used by microservice instances



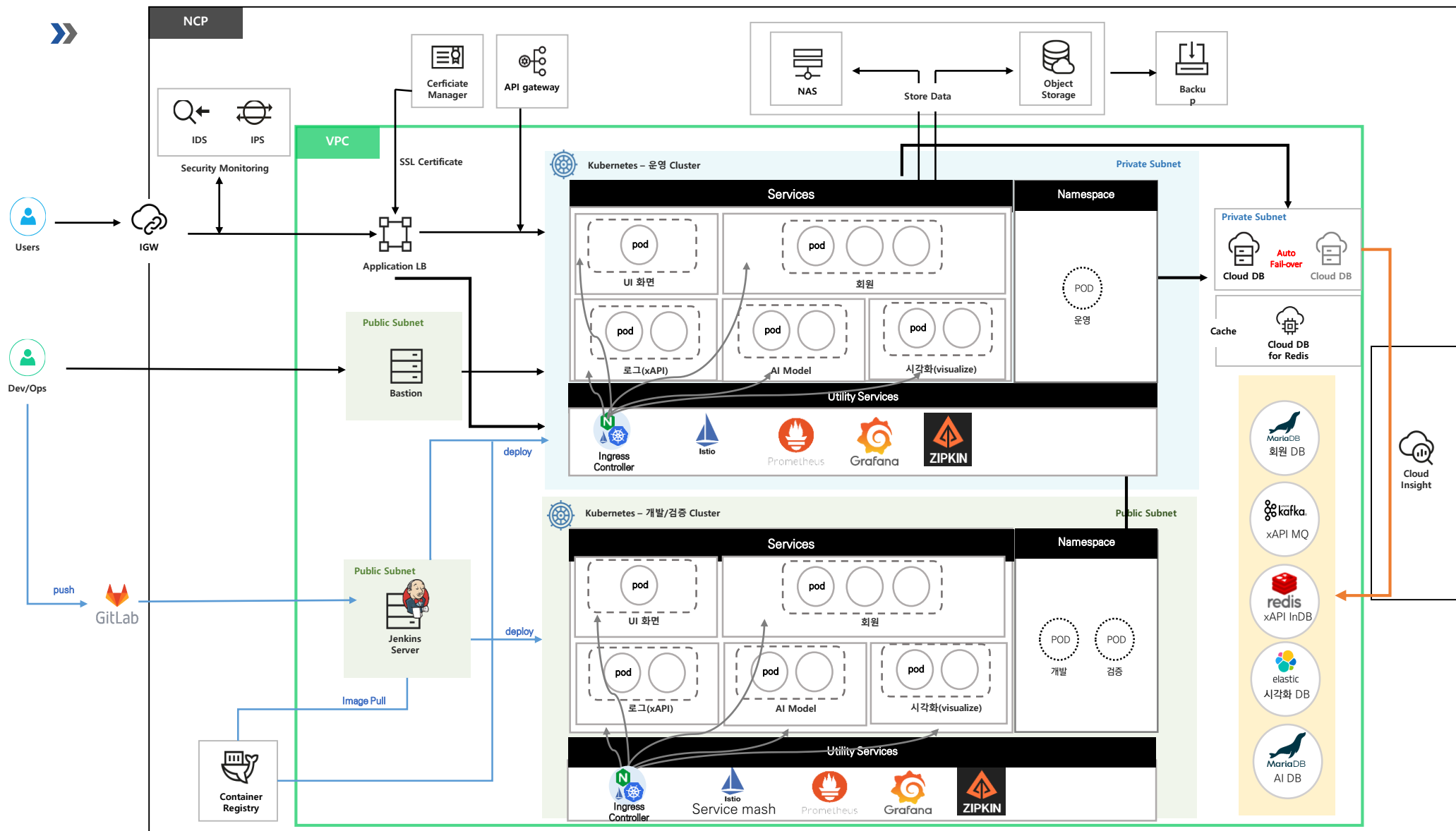
What are cloud native applications?

Design pattern	Spring boot	Spring cloud	kubernetes	Istio
Service search		Eureka and Ribbon	Kube-proxy, service	
Edge server		Spring cloud G/W, Oauth	Ingress Controller	Istio Ingress G/W
Reactive MS	Spring reactor and webclient			
Centralized config		Spring config	Configmap and Secret	
Centralized log		ELK	ELK, FluentD	
Distributed trace		Slueth and Zipkin		Jaeger
Circuit breaker		Hystrix, Resilience4j		Detect anomaly
Control loop			Kubernetes controller manager	
Monitoring and alarm			Grafana, Prometheus	Kiali, Grafana, Prometheus

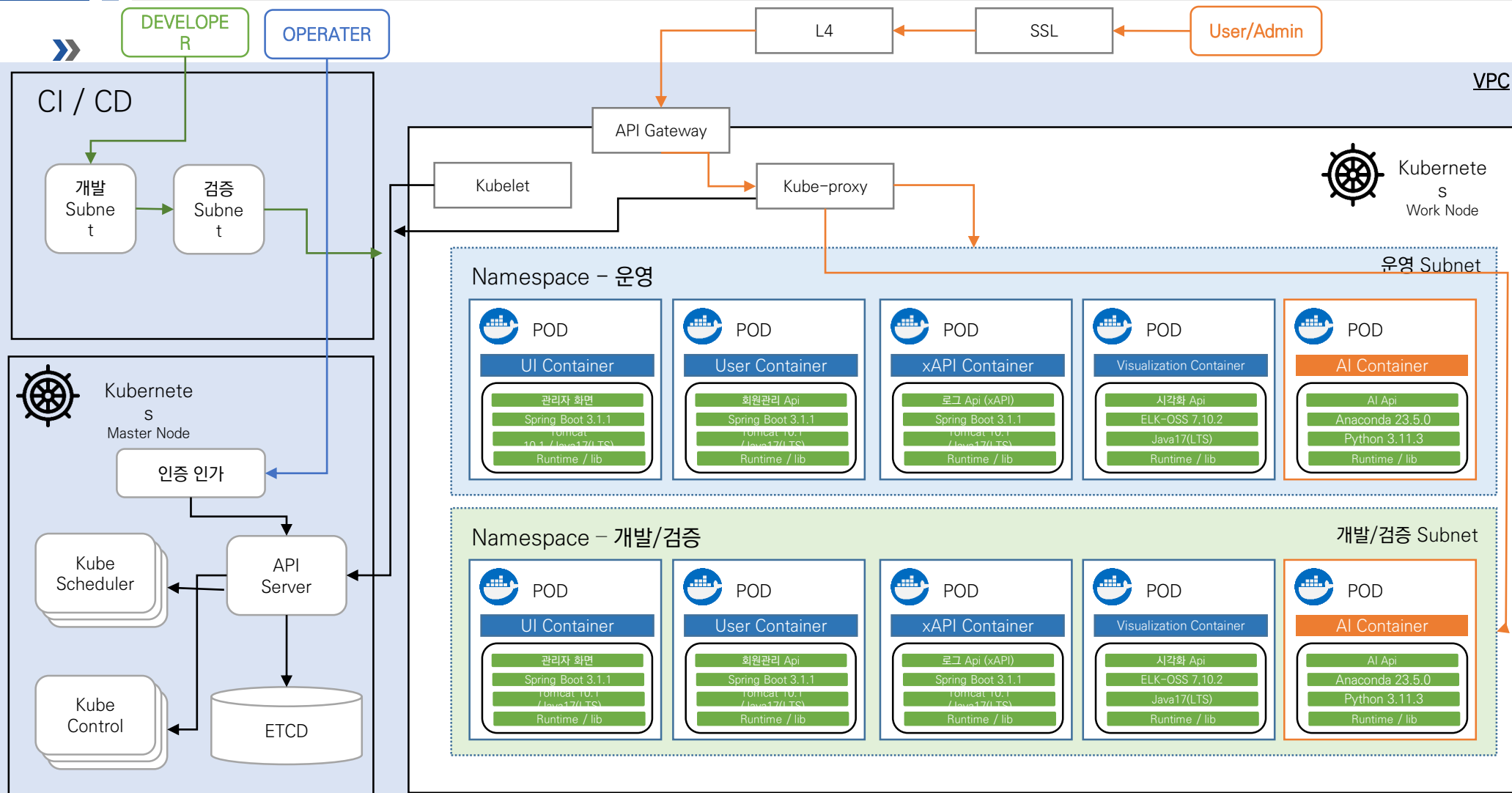
A top-down view of a wooden desk with a laptop, glasses, a cup of coffee, and a small plant. The text "04. MSA Configuration Examples" is overlaid in the center.

04. MSA Configuration Examples

Configuration Case 1

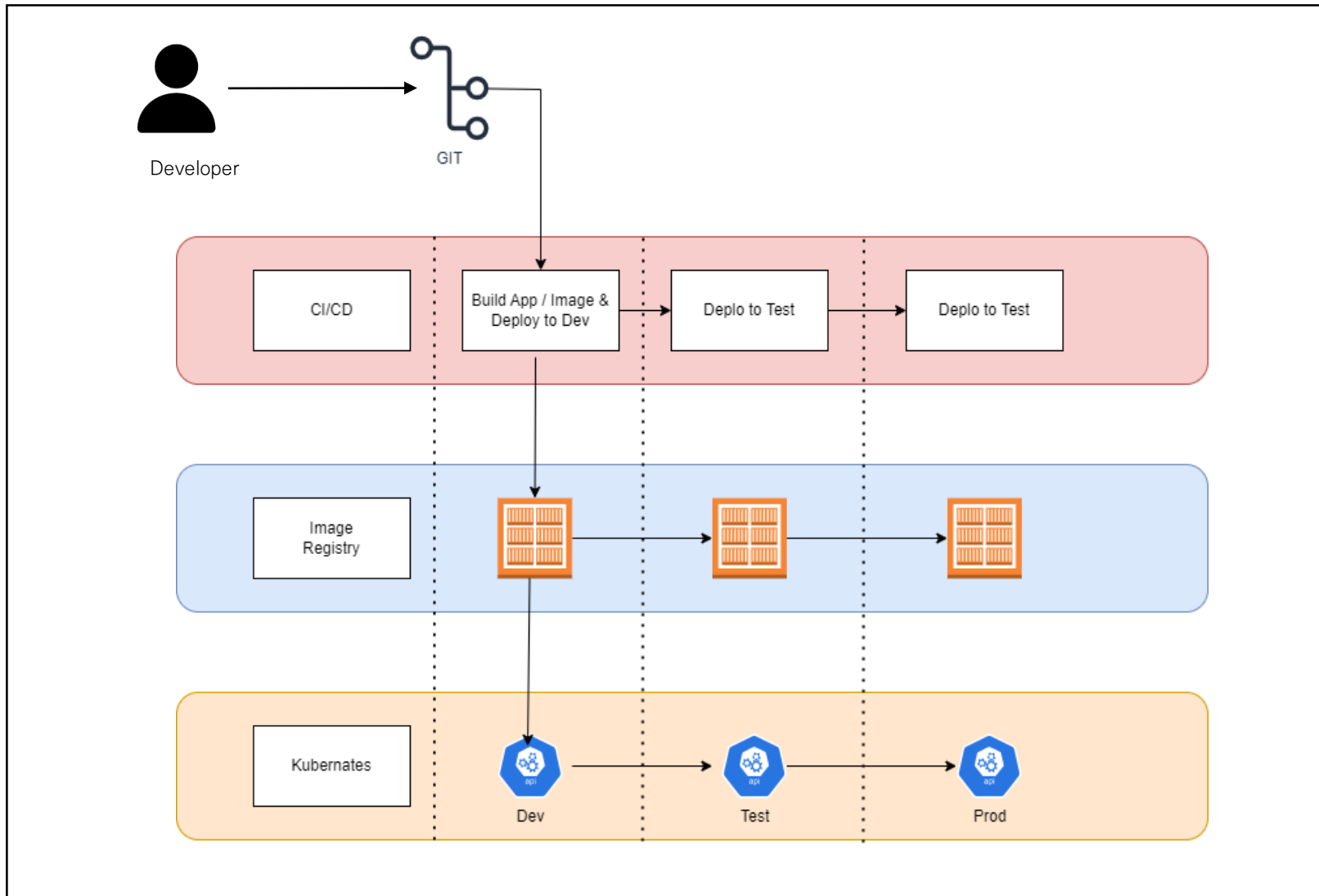


Configuration Case 2



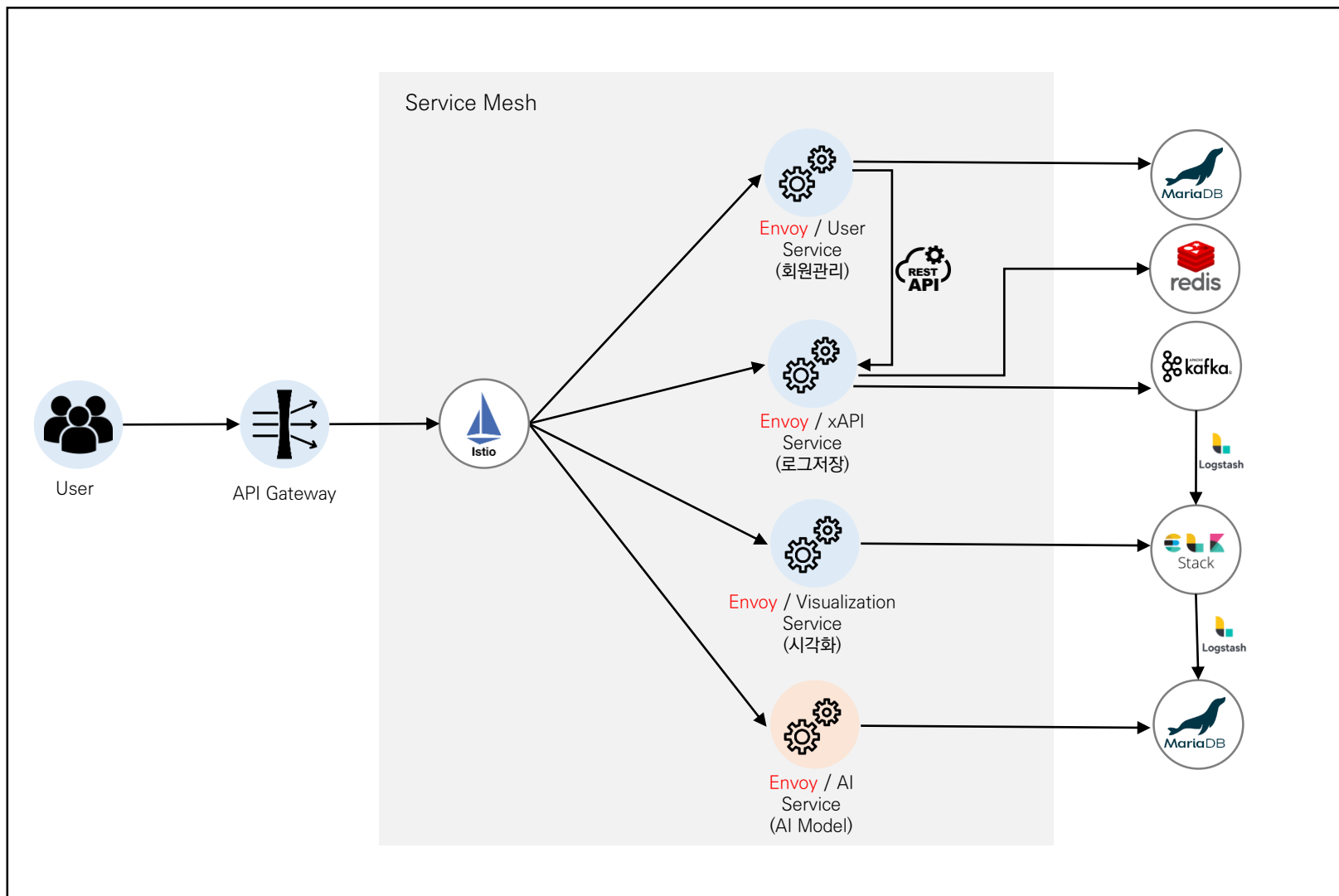
Configuration Case 3

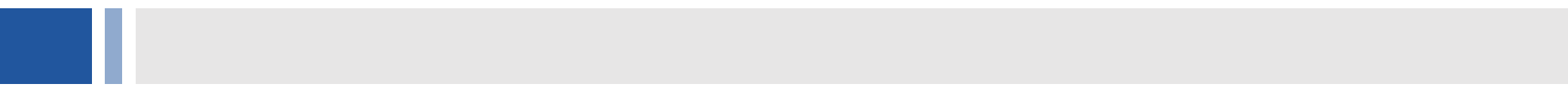
» CI / CD Environment



Configuration Case 1

» Call Relationships Between Services





감사합니다

THANK YOU